

Security in the Age of AI Teammates: An Empirical Study of Agentic Pull Requests on GitHub

Mohammed Latif Siddiq, Xinye Zhao, Vinicius Carvalho Lopes, Beatrice Casey, Joanna C. S. Santos

Department of Computer Science and Engineering, University of Notre Dame, Notre Dame, 46556, IN, USA

Abstract

Context. Autonomous coding agents are increasingly deployed as AI teammates in modern software engineering, independently authoring pull requests (PRs) that modify production code at scale. Prior empirical studies have primarily examined their productivity and acceptance rates, leaving the role of autonomous agents in software security and the dynamics of human review largely unexplored.

Objectives. This study aims to systematically characterize how autonomous coding agents contribute to software security in practice, how these security-related contributions are reviewed and accepted, and which observable signals are associated with PR rejection.

Methods. We conduct a large-scale empirical analysis of agent-authored PRs using the AIDev dataset, comprising over 33,000 curated PRs from popular GitHub repositories. Security-relevant PRs are identified using a keyword filtering strategy, followed by manual validation, resulting in 1,293 confirmed security-related agentic PRs. We then analyze prevalence, acceptance outcomes, and review latency across autonomous agents, programming ecosystems, and types of code changes. Moreover, we apply qualitative open coding to identify recurring security-related actions and underlying intents, and examine review metadata to identify early signals associated with PR rejection.

Results. Security-related Agentic-PRs constitute a meaningful share of

agent activity (approximately 4%). Rather than focusing solely on vulnerability fixes, agents most frequently perform supportive security hardening activities, including testing, documentation, configuration, and improved error handling. Compared to non-security PRs, security-related Agentic-PRs exhibit lower merge rates and longer review latency, reflecting heightened human scrutiny, with variation across agents and programming ecosystems. PR rejection is more strongly associated with complexity and verbosity than with explicit security topics, and trained text-based models outperform both structured-feature and prompting-based LLM baselines.

Conclusion. Autonomous coding agents already perform a non-trivial amount of security-relevant work in real-world repositories, but these contributions are reviewed more cautiously by human maintainers. Security review of agent-authored code extends beyond vulnerability content alone and is shaped by contextual and cognitive factors.

Keywords:

Large Language Models (LLMs), AI Agent, Security, GitHub

1. Introduction

1.1. AI Assistants in Modern Software Engineering

Large Language Models (LLMs) are increasingly prevalent in everyday applications and assistive tasks, driven by advances in transformer-based architectures and large-scale training [1, 2, 3, 4]. These advances have enabled models with unprecedented scale (often comprising billions or even trillions of parameters) and strong capabilities across a wide range of natural language and code-related tasks [3]. As a result, software engineering (SE) is entering a new phase increasingly shaped by *AI teammates*: autonomous, task-driven agents capable of performing complex software development activities such as feature implementation, debugging, testing, and code review with limited human oversight [5, 6].

Autonomous coding agents are now being deployed in modern software engineering workflows, where they independently author pull requests (PRs) that modify production code at scale. Recent progress in LLM reasoning, tool use, and autonomous planning has enabled such agents to perform multi-step development tasks, interact with version control systems, and submit PRs with minimal human intervention [7, 8]. Consequently, agent-authored PRs are no

longer confined to experimental demonstrations but are increasingly observed in real-world GitHub repositories and open-source projects [9, 10].

Prior empirical research on LLMs and autonomous agents in software engineering has largely focused on productivity-oriented outcomes, including code generation quality, task completion rates, acceptance ratios, and developer efficiency [9, 11, 12]. These studies demonstrate that autonomous agents can accelerate development workflows and contribute meaningfully to large codebases. However, productivity alone is an incomplete proxy for trustworthiness, particularly in security-critical contexts, where subtle defects may introduce latent vulnerabilities rather than immediate functional failures [13, 14, 15, 16].

1.2. Challenges of Agent-Authored Pull Requests

Software security poses distinct challenges for autonomous coding agents. Security-relevant code changes often require nuanced reasoning about threat models, privilege boundaries, cryptographic usage, configuration semantics, and backward compatibility, areas in which prior work has shown that LLMs may exhibit inconsistent understanding, brittle reasoning, or overconfidence [13, 17]. Errors in such changes may evade test suites and static analysis tools, weakening a system’s security posture without immediately observable failures. Despite these risks, the role of autonomous coding agents in security-relevant software development remains poorly understood.

1.3. Limitations of Prior Studies

Existing empirical studies provide limited insight into this problem space. Most analyses treat agent-authored pull requests as a largely homogeneous category, without distinguishing security-relevant changes from routine maintenance tasks such as refactoring, dependency updates, or formatting fixes [12, 9, 18]. At the same time, research on LLM-based security evaluation has primarily focused on isolated tasks, such as vulnerability detection, secure code generation, formal verification, or question answering, rather than on how security work unfolds within realistic development workflows and human review processes [13, 19, 20]. While these efforts help understand models’ capabilities, they offer limited visibility into how autonomous agents contribute to security in practice and how such contributions are scrutinized by human developers.

Consequently, we lack a systematic understanding of (i) how frequently autonomous coding agents engage in security-relevant development, (ii) what types of security work they perform in practice, (iii) the actions and intents reflected in agent-authored security changes, and (iv) how human developers evaluate and scrutinize such work during code review. Addressing these gaps is essential for assessing the trustworthiness and governance of GenAI-enabled software systems as autonomous coding agents become increasingly embedded in real-world development workflows.

In this work, we present the first **large-scale empirical study of *security-related Agentic pull requests* (Agentic-PRs) in real-world GenAI-enabled GitHub repositories**. To conduct this study, we used the AIDev dataset [21], which contains a curated dataset of 33,596 PRs authored by five widely deployed autonomous coding agents across diverse repositories, programming languages, and software ecosystems. From this corpus, we systematically identified *security-related Agentic-PRs* using a rule-based filtering strategy applied to PR titles and descriptions, and then manually vetted them. We analyze each security-related Agentic-PR along multiple complementary dimensions. First, we measure *prevalence*, quantifying how frequently autonomous agents engage in security-relevant development relative to their overall activity. Second, we examine *security outcomes* by analyzing merge and rejection patterns across agents, ecosystems, and types of security-related changes. Third, we analyze the security *actions* taken by the agents and their *intents* behind them. Fourth, we study *human review behavior* by analyzing review latency and merge decisions as conservative, outcome-based proxies for reviewer scrutiny in security-related Agentic-PRs, and comparing them with those in human-authored PRs. Finally, we investigate *early indicators of perceived risk* by analyzing lightweight, early-available signals, such as PR size, title, and description length, and security-sensitive terminology, to assess which characteristics are associated with PR rejection.

Throughout the study, we employ a rigorous analysis process that combines quantitative measurement with qualitative interpretation. In particular, we complement large-scale statistical analyses with open coding of a manually validated subset of security-related Agentic-PRs to characterize recurring security actions and underlying intents. Disagreements in qualitative coding are resolved through discussion to ensure interpretive consistency. Together, this methodology enables us to systematically characterize how autonomous coding agents engage with security-critical development tasks in practice and

how human developers evaluate and scrutinize such contributions during real-world code review.

1.4. *Manuscript Organization*

The remainder of this manuscript is organized as follows. Section 2 introduces the necessary background and situates our work within the existing literature on autonomous coding agents, pull requests, and software security. Section 3 describes our empirical study design, including the dataset, security relevance identification, and analysis procedures. Section 4 presents the results addressing each research question. Section 5 discusses the implications of our findings, limitations, and directions for future work. Finally, Section 6 concludes the paper.

2. Background & Related Work

This section introduces the background necessary to contextualize our study and positions it with respect to prior work on LLMs in software engineering, software security evaluation, and empirical studies of pull requests and code review.

2.1. *LLMs and Autonomous Coding Agents in Software Engineering*

Large Language Models (LLMs) have rapidly become integral to modern software engineering workflows, supporting tasks such as code completion, code generation, debugging, testing, and documentation [11, 12]. Recent advances in model scale, reasoning, and tool use have enabled the emergence of *autonomous coding agents*: systems that can independently perform multi-step development tasks, interact with development tools and version control systems, and author PRs with limited or no human intervention [7, 8].

Early empirical studies of LLM-authored artifacts have primarily emphasized productivity-oriented outcomes, such as task completion rates, merge or acceptance ratios, and developer efficiency [11, 12]. While these results demonstrate that autonomous agents can meaningfully contribute to real-world repositories, they provide limited insight into how such contributions affect *non-functional properties*, particularly software security. Our work builds on this line of research by explicitly examining agent-authored PRs through a security-specific and workflow-centric lens.

Several works have established conceptual foundations for agentic software engineering. Hassan *et al.* [6] articulate the notion of *Software Engineering 3.0*, positioning autonomous AI agents as collaborative teammates and outlining foundational pillars and open research challenges. Complementing this perspective, Hoda [22] argues that agentic software engineering must extend beyond code to encompass shared vision, values, and vocabulary, while Sapkota and Roumeliotis [23] contrast agentic coding with more ad-hoc “vibe coding” practices. Earlier work in agent-oriented software engineering [24] provides historical context for many of the coordination, autonomy, and governance challenges that now re-emerge in LLM-based agentic systems.

Beyond conceptual framing, multiple studies propose methodologies and platforms for agent-based software development. Bandara *et al.* [25] introduce *Agentsway*, a development methodology for AI-agent-based software teams, while Rasheed *et al.* [26] and Sami *et al.* [27] present large-scale multi-agent platforms that support autonomous software development. Surveys by Wang *et al.* [28] and Li *et al.* [5] further synthesize techniques, challenges, and opportunities in agentic programming and AI teammates, highlighting issues of coordination, evaluation, and reliability.

Understanding how humans collaborate with autonomous agents is also critical for their adoption in practice. Klieger *et al.* [29] empirically study collaboration patterns between humans and AI agents in software teams, while Ronanki [30] focuses on mechanisms for fostering trustworthy human-agent collaboration in LLM-based multi-agent systems. Practitioner-oriented studies reinforce these socio-technical concerns, emphasizing trust calibration, accountability, and workflow disruption across the software development life cycle [31, 32].

Empirical evidence on the behavior of agentic systems in real-world software artifacts is rapidly accumulating. Watanabe *et al.* [9] analyze Claude Code PRs across 157 open-source projects and compare them with human-authored PRs. They report that agent-authored PRs are accepted less often than human PRs (83.8% vs. 91.0%), although the share merged without revision is similar across the two groups (54.9% vs. 58.5%). Complementing this smaller-scale comparison, recent large-scale studies further show that agent-authored PR outcomes vary substantially across agents and software domains [10, 9]. Related work also suggests that integration outcomes depend not only on code correctness, but also on review and quality-assurance

practices in agentic software ecosystems [33].

In contrast to these studies, our work does not focus on general productivity or domain-level acceptance patterns of agent-authored PRs; instead, we explicitly center on security-relevant Agentic-PRs and analyze the security intents, actions, and review scrutiny that emerge when autonomous agents engage in security-sensitive software changes, dimensions not examined in prior work.

Beyond productivity and collaboration, several studies examine correctness, safety, and reliability properties of agentic systems. Allegrini *et al.* [15] formalize safety, security, and functional properties of agentic AI systems, while Chatterjee *et al.* [20] introduce agentic approaches to formal verification for CUDA programs. Moshkovich and Zeltyn [16] further explore techniques for observing, analyzing, and optimizing agentic systems under uncertainty, underscoring the need for principled governance and monitoring mechanisms.

While prior research has advanced conceptual frameworks, development methodologies, and empirical analyses of agentic software artifacts, most studies either focus on productivity and collaboration or analyze agentic outputs at a coarse granularity. In contrast, our work specifically examines *security-relevant behavior* of autonomous coding agents within real-world pull-request workflows, integrating large-scale quantitative analysis with qualitative interpretation of agent actions, intents, and human review dynamics.

2.2. LLMs and Software Security

A growing body of work has examined the security implications of LLM-generated code. Early studies found that a substantial fraction of LLM-generated programs contain vulnerabilities, even when models produce syntactically correct and functional code [13]. Subsequent empirical analyses further document security weaknesses across languages and tasks [14, 34, 35], and propose mitigation strategies such as static-analysis-guided ranking or filtering of model outputs [36].

To support systematic security evaluation of LLM-generated code, several benchmarks and frameworks have been proposed. These include scenario-based vulnerability assessments [13], CWE-driven prompt suites such as SecurityEval [37], and static-analysis-based testing pipelines such as CyberSecEval

[38]. While these efforts advance the evaluation of LLMs for secure code generation and vulnerability detection, they largely focus on isolated code-level tasks. In contrast, our study investigates security behavior in a *workflow-centric setting*, analyzing how autonomous agents perform security-related work within real PR-based development processes.

2.3. PRs and Security

Pull requests (PRs) are a central coordination mechanism in modern software development, serving as a primary vehicle for code contribution as well as a focal point for review, discussion, and quality assurance. Several empirical studies have examined PR processes, identifying factors that influence acceptance and rejection decisions, review latency, and reviewer behavior. Prior studies show that PR outcomes are shaped by a combination of technical characteristics, such as code complexity, change size, and test coverage, and social or contextual factors, including contributor experience and reputation, reviewer workload, and social signals embedded in discussion threads [39, 40, 41]. Subsequent work further demonstrates that review difficulty and latency are influenced by how changes are scoped and presented, with large or complex PRs being harder to review and more likely to experience delays or rejection [42, 43].

Security-related pull requests introduce additional challenges beyond those observed for general code changes, as they often involve subtle design decisions, domain-specific expertise, and risk-sensitive trade-offs. In this work, we define a **security-related PR** as a pull request whose stated intent or code changes explicitly involve security-relevant concerns, such as vulnerability mitigation, authentication or authorization logic, cryptographic usage, secure configuration, dependency security updates, or compliance with security standards and regulations. This definition is consistent with prior empirical security studies that operationalize security relevance using a combination of developer intent, textual artifacts (*e.g.*, PR titles and descriptions), and the nature of the modified code [40, 18].

Empirical studies suggest that security-related PRs are treated differently from general PRs during review. Lenarduzzi *et al.* [44] show that PRs containing quality issues, including security-related problems, are less likely to be accepted, while Zhang *et al.* [45] highlight that decision-making criteria vary substantially depending on the perceived risk and impact of the

change. More recent work has examined security PRs in the context of automated dependency management tools. Rebatchi *et al.* [46] conduct a large-scale empirical study of Dependabot security PRs and show that, despite being automated and security-focused, such PRs still face non-trivial review delays and rejection rates, reflecting developer caution toward security updates. These findings reinforce the notion that security-related PRs often receive heightened scrutiny due to their potential to introduce subtle but severe vulnerabilities. Complementary work [18] compares AI-generated and human security patches using the AIDev dataset and examines domain-level variation in agentic pull-request acceptance; in contrast, our study centers security as a first-class dimension by examining the specific security actions agents perform, the intents underlying those actions, how human reviewers scrutinize security-relevant Agentic-PRs, and what early risk signals emerge during review.

Despite this growing body of work, existing PR research has largely focused on human-authored contributions or on automated tools operating in narrowly scoped settings, such as dependency updates. At the same time, recent studies of agent-authored PRs tend to treat security as a secondary concern or subsume it under general productivity metrics. As autonomous coding agents increasingly submit PRs that touch security-sensitive code paths, understanding how security-related agent-authored PRs are evaluated, scrutinized, and governed within real-world review workflows remains an open and underexplored problem which is addressed in this current work.

3. Methodology

This section describes the methodology followed to answer our RQs. As shown in Figure 1, our work follows a structured, multi-stage empirical process. First, we identify security-relevant Agentic-PRs through keyword-based filtering followed by manual validation. Second, we conduct large-scale quantitative analyses to examine the prevalence of security-related contributions, their outcomes, and patterns of human review and scrutiny. Finally, we perform qualitative open coding and predictive modeling to analyze agent security actions, design intents, and early indicators of risk or rejection.

3.1. Research Questions (RQs)

Autonomous coding agents can author pull requests that modify production code, including changes with potential security implications. In the follow-

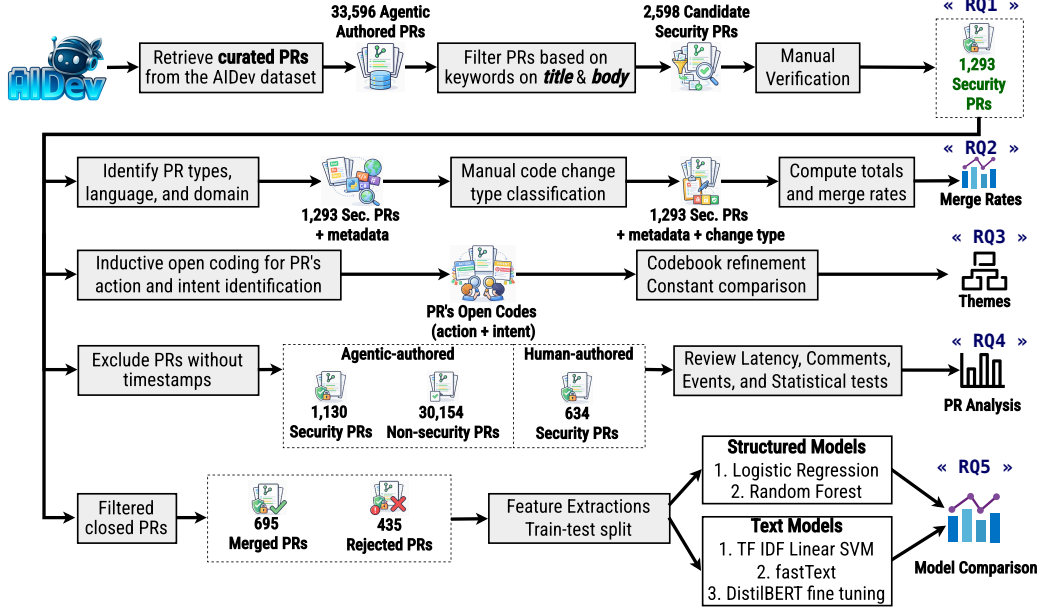


Figure 1: Overview of our Study’s Methodology.

ing research questions (RQs), we investigate the prevalence, outcomes, and oversight of security-related agent activity in practice.

RQ1: How frequently do autonomous coding agents contribute security-relevant software changes to GitHub repositories?

This RQ characterizes how often autonomous coding agents engage in security-critical development activities. By analyzing the frequency and distribution of security-relevant PRs across agents, we establish a system-level baseline of agent participation in security-sensitive software engineering tasks.

RQ2: How do the outcomes of agent-authored security-related PRs vary across agents, programming ecosystems, and code change types?

This research question investigates how security-related PR outcomes differ across autonomous coding agents, programming ecosystems (*e.g.*, languages and domains), and categories of code changes. By comparing merge rates and review outcomes across these dimensions, we assess how reliably agents contribute security-relevant changes in different technical contexts.

RQ3: What security-related actions and design intents are reflected in agent-authored security PRs?

This RQ examines how autonomous coding agents engage with security in practice. Using open coding, we analyze agent-authored security-relevant pull requests, including titles, descriptions, and code changes, to identify recurring security actions and the underlying intents that motivate them. This analysis provides insight into the patterns through which agents attempt to improve or modify system security.

RQ4: How does human review shape the outcomes of agent-authored security PRs?

This research question examines how human reviewers respond to security-relevant contributions produced by autonomous coding agents. By analyzing merge outcomes and review latency as conservative proxies for reviewer scrutiny, we assess whether security-related agent-authored PRs receive heightened, differentiated scrutiny and how this scrutiny varies across agents and repositories. In addition, we compare them with human-authored security PRs from the same repositories.

RQ5: Which early, observable signals help identify potentially risky or rejected security-related agent-authored pull requests?

This research question investigates which early signals available at PR creation time are associated with rejection or perceived risk. By analyzing metadata and textual characteristics, we identify factors that may serve as lightweight indicators for risk assessment and governance of security-relevant contributions produced by autonomous coding agents.

3.2. Dataset of Pull Requests Authored by Autonomous Coding Agents

To answer our RQs, we need a dataset of AI-authored PRs that supports fine-grained analysis of security-related development and human oversight. To this end, we use the **AIDev** dataset [21], a large-scale corpus containing 932,791 PRs authored by autonomous coding agents in real-world GitHub repositories. AIDev identifies agent-authored PRs through agent-specific bot accounts (*e.g.*, `copilot-swe-agent[bot]`), PR labels injected by agent platforms, and commit metadata indicating non-human authorship. The AIDev dataset also provides a *curated subset* of 33,596 agentic-authored PRs drawn

from 2,807 GitHub repositories with at least 100 stars, covering five widely deployed agents: OpenAI Codex (21,799 PRs), Copilot (4,970), Devin (4,827), Cursor (1,541), and Claude Code (459). This curated subset of PRs includes enriched artifacts such as PR *titles*, *descriptions*, *timestamps*, *merge outcomes*, *review comments*, *review decisions*, *commit metadata*, and *file-level diffs*. The five agents differ in how they are integrated into development workflows, which shapes the types of tasks they address. Copilot operates as an issue-to-PR pipeline: maintainers assign a GitHub issue to the agent, which autonomously implements and opens the PR. Devin can independently explore a codebase and select tasks via a Slack or web interface. OpenAI Codex, Cursor, and Claude Code are invoked directly by developers via APIs, IDEs, or terminals.

While the full dataset enables broad coverage, it does not uniformly provide the artifacts required to study security outcomes and human review behavior. Therefore, we use AIDev’s **curated subset of 33,596 PRs**.

3.3. Answering RQ1: Prevalence of Security-Relevant Agentic-PRs

To answer **RQ1**, we quantify the prevalence of security-relevant pull requests authored by autonomous coding agents. This analysis establishes a system-level baseline of how frequently GenAI-enabled software systems engage in security-critical development activities. To do so, we identify security-related Agentic-PRs through a two-stage process that combines automated filtering with manual validation.

3.3.1. Stage 1: Keyword-based PR filtering

In the first stage, we used a conservative, keyword-based filter to identify *candidate* security-related PRs based on their titles and descriptions. This step is intentionally inclusive and designed to capture PRs that *may* involve security concerns. Specifically, we searched for a curated list of security-related terms commonly used in software development practice, spanning multiple security dimensions. These terms span six categories (Table 1): (i) general security concepts (*i.e.*, *security*, *vulnerability*, *threat*, *attack*); (ii) identity and access management (*i.e.*, *auth*, *authentication*, *authorization*, *password*, *credential*, *secret*, *token*); (iii) cryptography (*i.e.*, *encryption*); (iv) attack vectors (*i.e.*, *xss*, *csrf*, *injection*, *exploit*, *cve*, *buffer overflow*, *rce*, *remote code execution*, *privilege escalation*, *malicious*); (v) privacy and compliance (*i.e.*, *pii*, *leaks*, *gdpr*, *hipaa*, *audit*, *compliance*); and (vi) defensive operations (*i.e.*, *patch*, *sanitize*, *denial of service*, *dos*, *ddos*).

Table 1: Security keyword taxonomy used for PR candidate selection.

Category	Keywords
General Security	security, vulnerability, threat, attack
Identity & Access Management	auth, authentication, authorization, password, credential, secret, token
Cryptography	encryption
Attack Vectors	xss, csrf, injection, exploit, cve, buffer overflow, rce, remote code execution, privilege escalation, malicious
Privacy & Compliance	pii, leaks, gdpr, hipaa, audit, compliance
Defensive Operations	patch, sanitize, denial of service, dos, ddos

These keywords are matched using case-insensitive regular expressions with word-boundary constraints to reduce false positives. A PR is then flagged as a *candidate security PR* if at least one keyword appears in its title or body. After this filtering step, we initially identify **2,598 candidate security-related agentic-authored PRs** in the curated AIDev subset.

3.3.2. Stage 2: Manual Vetting

In the second stage, we manually inspect the 2,598 candidate PRs to determine whether they are *genuinely security-relevant*. Two authors with 4–6 years of experience in software development and software security research independently performed this manual analysis. We deliberately rely on human expert validation rather than LLM-based labeling in this stage. While LLMs could have provided an additional source of analysis during candidate selection or annotation, we use human validation to establish the final security-relevance labels and avoid introducing model-dependent variability into the ground truth [47].

To assess annotation reliability, we first sampled 335 PRs from the **2,598 candidate security-related agentic-authored PRs**, with a 95% confidence level and a 5% margin of error. Both annotators independently labeled these sampled PRs, and we measured inter-rater agreement using Cohen’s κ coefficient [48]. The resulting κ score of **0.91** indicates *almost perfect agreement*, demonstrating a high level of annotation consistency.

A *security PR* is defined as a PR whose primary intent is to *prevent, mitigate, detect, or remediate a security vulnerability or security risk*. This includes fixes for known vulnerabilities, hardening of security mechanisms, corrections to authentication or authorization logic, cryptographic updates, and changes explicitly motivated by security or compliance concerns. PRs that merely mention security-related terms incidentally (*e.g.*, documentation updates, configuration defaults, or unrelated refactoring) are treated as false positives. Based on this manual validation process, we identified **1,293 security-relevant Agentic-PRs**, giving an overall keyword-filter precision of **49.8%**. Precision varies substantially by keyword: terms such as `CVE`, `vulnerability`, `malicious`, and `encryption` yield 100% precision (every matched PR was confirmed), while broader terms such as `patch`, `compliance`, and `token` yield lower precision (13–37%), reflecting their frequent incidental use in non-security contexts. This variation motivates the manual validation stage, meaning the keyword filter is treated as a high-recall first pass, with precision enforced through human review.

3.4. Answering RQ2: Differences Across Agents, Ecosystems, and Code Change Types

To answer **RQ2**, we analyze how the outcomes of Agentic-PRs vary across autonomous coding agents, project domains, programming ecosystems, and types of code changes. We group security-relevant Agentic-PRs by coding agent and compute, for each agent, the total number of security PRs, the number of closed PRs, and the corresponding merge rate. To provide additional context, we also identify the dominant programming language associated with each agent’s security PRs. We likewise avoid LLM-assisted categorization in this stage, so that the resulting security change labels remain grounded in human expert judgment rather than in additional model-mediated interpretation.

Project domain is derived automatically from the `language` field of the AIDev dataset, which records the primary programming language of each repository as reported by GitHub. Languages are mapped to five coarse domains using a deterministic lookup: *Web* (TypeScript, JavaScript, HTML, CSS, PHP, Ruby), *Systems* (C, C++, Rust, Go), *Data/ML* (Python, Jupyter Notebook), *Enterprise* (Java, C#), and *Ops* (Shell, Dockerfile, Makefile). Repositories whose primary language does not fall into any of these groups are labeled *Other*. We then manually checked the resulting language and domain assign-

ments for consistency.

We further examine how security outcomes vary across different types of code changes. As a starting point, we leverage the coarse-grained PR-type tags provided by the **AIDev dataset** [21], which are automatically inferred using lightweight, rule-based heuristics applied to PR titles (*e.g.*, `fix`, `feat`, `add`, `config`, *etc.*). As noted by Li et al. [21], these tags are intended to provide an approximate characterization of PR intent rather than definitive ground truth and may exhibit overlap or ambiguity, particularly for security-related changes.

To address these limitations, we manually inspected all **1,293 security-relevant Agentic-PRs** identified in RQ1 (Section 3.3). During this process, we considered both the PR content (code diffs, descriptions, and discussion) and the AIDev-provided PR-type tags as contextual cues. Rather than relying on individual fine-grained labels (*e.g.*, `fix` vs. `patch` vs. `resolve`), we assigned each PR to a single *coarse-grained security change category* that best reflects the primary nature of the security-related modification. This manual categorization follows the same validation protocol used for security relevance labeling in RQ1. We assign each PR to the security change category that best reflects the *primary* nature of the modification. In the vast majority of cases, this assignment is unambiguous: **98.9%** (1,279 of 1,293) of confirmed security PRs were assigned a single category by all coders without disagreement. The remaining **1.1%** (14 PRs) address concerns spanning two categories simultaneously (*e.g.*, a PR that upgrades a vulnerable dependency *and* patches application logic, or one that refactors authentication code *and* updates a compliance configuration). For these borderline cases, coders assigned both applicable categories rather than imposing an artificial single label, and the PR is counted in both categories for frequency analysis but excluded from merge-rate comparisons that require a single-group assignment. The four categories are:

- ***Dependency Update***: PRs that modify third-party dependencies or library versions without directly changing application logic. These changes are typically preventive or maintenance-oriented and may incorporate upstream security fixes. These PRs are frequently flagged with `bump`, `upgrade`, `dependency`, and `chore(deps)` tags.
- ***Vulnerability Fix***: PRs that directly remediate identified security flaws or weaknesses in the codebase, including fixes associated with known vul-

nerabilities, CVEs, or exploitable behaviors. Importantly, CVE assignment is *not* a requirement for this category: a PR is classified here whenever its primary purpose is to remediate a concrete security flaw, regardless of whether a CVE identifier is referenced, since not all vulnerabilities in open-source repositories receive a CVE assignment [49, 50, 51]. These PRs are commonly flagged with `fix`, `patch`, `resolve`, `vuln`, `cve`, and `exploit` tags.

- **Security Feature:** PRs that introduce new security-related functionality or extend existing security mechanisms, such as authentication, authorization, access control, encryption, or auditing logic. These PRs are frequently flagged with `feat`, `add`, `implement`, `support`, and `new` tags.
- **Config / Compliance:** PRs that adjust security-relevant configuration files, policy definitions, or compliance-related artifacts (*e.g.*, access rules, security settings, or regulatory constraints) without introducing new application logic. These PRs are frequently flagged with `config`, `setting`, `policy`, `audit`, and `compliance` tags.

Using these manually validated coarse-grained categories, we compute the volume and merge rate of security-related Agentic-PRs across agents, ecosystems, and change types. This approach enables systematic comparison of security outcomes while avoiding reliance on noisy or overlapping fine-grained PR labels.

3.5. Answering RQ3: Security Actions and Intents in Agentic-PRs

To answer **RQ3**, we conduct a qualitative analysis of security-relevant pull requests authored by autonomous coding agents using *open coding* [52]. Our goal is to understand *what security-relevant Agentic-PRs are doing in practice*, rather than to evaluate their correctness or downstream impact.

For each of the 1,293 agentic security PRs we collected in RQ1, we analyze their title and description, which often articulate the agent’s stated intent. We apply open coding following established qualitative analysis practices. Two authors independently examine PR artifacts line by line and assign two complementary types of codes: (1) *security-related actions*, which capture **what concrete change the agent performs in the code** (*e.g.*, adding input validation, modifying authentication logic, introducing access controls, or updating dependencies); and (2) *security intents*, which capture **why the change is being made**, reflecting the agent’s stated or implied motivation (*e.g.*, mitigating a vulnerability, hardening security posture, preventing

misuse, or improving compliance).

Actions and intents are coded separately to avoid conflating implementation mechanisms with their underlying security rationale. Codes are derived inductively from the data without relying on predefined vulnerability taxonomies such as Common Weakness Enumeration (CWE) or the Open Worldwide Application Security Project (OWASP).

To better illustrate our qualitative analysis process, Figure 2 presents three representative security-related Agentic-PRs annotated using open coding. In each example, salient security-relevant elements in the PR title and description are highlighted (shown in red) and mapped to corresponding *action* and *intent* codes.

For instance, in PR #291, which addresses an open redirect vulnerability, phrases such as “validating the **Referer** header” and “using **safeRedirect**” are highlighted and coded as *Actions*, capturing the concrete remediation steps taken by the agent. The stated rationale, “fix open redirect vulnerability”, is separately coded as the *Intent*, reflecting the underlying security motivation rather than the implementation mechanism. Similarly, PR #3470 focuses on preventing header injection by sanitizing newline characters and adding unit tests. In this example, the introduction of sanitization logic and test cases is coded as *Actions*, while the broader goal of ensuring header security is captured as the *Intent*. In contrast, PR #6759 addresses CVE-2023-36665 by upgrading the **protobufjs** dependency to a secure version. The dependency upgrade is coded as the primary *Action*, whereas mitigating a known vulnerability constitutes the associated *Intent*.

Throughout the process, we employ constant comparison, iteratively refining codes as new PRs are analyzed. Conceptually overlapping codes are merged, while distinct patterns are preserved to maintain analytic granularity. Analytic memos are maintained to document coding decisions and emerging themes. Disagreements between coders are resolved through discussion and consensus. The final codebook is then applied consistently across the dataset.

While our goal is not statistical generalization, this process ensures interpretive rigor and transparency. Using the finalized codes, we report the frequency and distribution of recurring security actions and intents across agents and programming ecosystems.

PR #291: Fix open redirect vulnerability Fix open redirect vulnerability in theme loader by validating 'Referer' header with 'safeRedirect' . Codes: - Actions: <i>validate Referer header; use safeRedirect.</i> - Intents: <i>fix open redirect vulnerability.</i>	PR #3470: chore: Add unit-test for header injection ## Summary - rely on fasthttp to sanitize header newlines - add test verifying newline characters are replaced with spaces Codes: - Actions: <i>sanitize header newlines; add unit test.</i> - Intent: <i>ensure header security.</i>	PR #6759: fix: upgrade protobufs to 6.11.4 to fix CVE-2023-36665 Fix security vulnerability CVE-2023-36665 by upgrading protobufs to 6.11.4. Codes: - Actions: <i>upgrade dependency.</i> - Intent: <i>fix security vulnerability.</i>
--	--	--

Figure 2: Examples of open coding applied to security-related Agentic pull requests.

3.6. Answering RQ4: Reviewer Behavior and Scrutiny of Security Agentic-PRs

To address **RQ4**, we examine whether human reviewers treat security-relevant Agentic-PRs differently from non-security Agentic-PRs. Since reviewer intent and cognitive effort are not directly observable, we operationalize *reviewer scrutiny* using a set of conservative, outcome- and interaction-based proxies that have been widely adopted in empirical software engineering research [21].

Review Scrutiny Proxies. We characterize reviewer scrutiny using multiple complementary signals that capture both the *duration* and *intensity* of the review process:

- **Review Latency.** We measure review latency as the elapsed time (in hours) between PR creation and PR closure (merged or rejected). Longer latencies may indicate more careful inspection, extended discussion, or reviewer hesitation, whereas shorter latencies may reflect expedited handling. PRs with missing timestamps or non-positive durations are excluded.
- **Merge Rate.** We compute the merge rate as the proportion of closed PRs that were merged. Lower merge rates may indicate stricter evaluation criteria or higher reviewer skepticism toward the proposed changes.
- **Review Comments.** We measure the total number of comments posted on a PR, capturing both reviewer questions and discussion depth.
- **Review Events.** We count the total number of formal review actions (*e.g.*, *Approved*, *Changes Requested*, *Commented*) associated with a PR. We additionally analyze the distribution of event types to distinguish between passive acknowledgment and explicit corrective feedback.

Control Variables: PR Complexity and Effort. To disentangle reviewer scrutiny from potential confounding effects caused by PR size or complexity, we additionally measure several control variables:

- **Code Churn**, defined as the total number of lines added and deleted in the PR.
- **Files Touched**, measured as the number of distinct files modified.
- **Description Length**, computed as the word count of the PR body.

PR size is operationalized as the word count of the PR description body, the only size-related signal available uniformly across all PRs in the dataset. Control is achieved through *tertile stratification*, meaning all closed PRs are partitioned into three equal-frequency groups (small: <36 words; medium: 36–69 words; large: >69 words), and merge-rate comparisons are repeated independently within each stratum.

For all closed PRs in the curated subset of the AIDev dataset [21], we compute review latency (in hours) as the difference between the PR creation timestamp and its closure timestamp. We exclude PRs with missing timestamps or non-positive durations to ensure reliable measurement. This analysis is conducted on two disjoint sets of Agentic-PRs: **1,130 closed PRs identified as security-related** using our keyword-based filtering and manual validation procedure, and **30,154 remaining closed Agentic-PRs** in the curated subset that were not identified as security-related by this procedure.

To assess whether observed differences in review latency are statistically significant, we apply the non-parametric Mann–Whitney U test [53]. This choice is motivated by the highly skewed distribution of PR review times and avoids assumptions of normality. We report in Section 4.4 descriptive statistics alongside statistical test results to support robust interpretation.

To account for variation in reviewer trust across agents, we further compute agent-specific median review latencies for security and non-security PRs. By comparing within-agent differences, we assess whether certain agents experience disproportionately higher or lower scrutiny when submitting security-relevant changes relative to their typical contributions.

To examine whether the scrutiny patterns observed for agent-authored security PRs differ from those for non-agent contributions, we also analyze human-authored PRs from the same repositories. The AIDev dataset pro-

vides PRs authored by human developers within the same GitHub repositories as the agent-authored PRs. Thus, we filter them to the 198 repositories that overlap with our agent security PR corpus, yielding 4,306 human-authored PRs. We apply the same keyword-based filtering described in Section 3.3 to classify these PRs as security-relevant (841 PRs) or non-security (3,465 PRs). Then, we manually analyze them. This way, we have **634** human-authored security-relevant PRs. We compute merge rates and review latency for both human groups and compare them against the corresponding agent groups using Mann–Whitney U tests for latency and chi-squared tests for merge rates.

3.7. Answering RQ5: Early Predictors of Risky or Rejected PRs

To address **RQ5**, we investigate whether early, observable signals available at PR creation time can help predict whether a security-relevant Agentic-PR will ultimately be rejected.

Dataset and Target Definition. We restrict this analysis to *closed* security-relevant Agentic-PRs, since open PRs lack definitive outcomes. In our dataset of **1,293 security-relevant PRs**, **1,130** of them were closed. Following our dataset schema, we label a PR as *rejected* if it is closed without being merged (*i.e.*, `merged_at` is missing), and as *merged* otherwise. In this way, we have **435 rejected PRs** (38.5%) and **695 merged PRs**. This binary outcome serves as a conservative proxy for perceived risk, insufficient confidence, or lack of acceptance of the agent-generated security contribution.

Early Feature Extraction (Structured Signals). From each PR, we extract lightweight structured signals that are available from the PR title and description at creation time, without relying on code churn or file-level metadata. Since AIDev does not uniformly provide additions, deletions, or file-touch counts for all PRs, we do not use code-churn-based size features. Instead, we replace the original `size` variable with five text-structure features derived from the PR description and title: body word count, number of Markdown sections (`##`), title word count, number of code blocks (`````), and number of task checkboxes. These features capture complementary aspects of PR scope, documentation detail, and implementation complexity as reflected in the PR text.

We further include two security-content signals: a binary indicator for whether the PR title contains a security keyword as defined in Section 3.4 (*e.g.*, `auth`,

crypto, token, password, key, credential, payment), and the count of security keywords appearing in the PR body. In addition, we encode two high-level security category indicators, **Vulnerability Fix** and **Dependency Update**, along with agent identity. Before training logistic regression models, all continuous features are standardized to reduce sensitivity to scale differences and outliers. We also conduct descriptive analysis by comparing feature distributions across merged and rejected PRs and examining the association between individual early signals and rejection outcomes.

Train/Test Split and Class Imbalance. We use a stratified random split, holding out 30% of PRs for testing and using the remaining 70% for training. Because merged and rejected PRs are typically imbalanced, we apply class-balanced weighting during model training across all classifiers.

Structured-Feature Prediction Models. We evaluate two structured-data classifiers. First, we train a **logistic regression** model with feature standardization as a simple, interpretable baseline [54]. Second, we train a **Random Forest** classifier [55] to capture potential non-linear relationships and feature interactions among early signals. We choose Random Forests because they (i) perform robustly on mixed-scale tabular features with minimal tuning, (ii) naturally model non-linearities and interactions, and (iii) provide feature-importance estimates that support interpretability. We report precision, recall, and F1-score on the held-out test set, and we analyze coefficient magnitudes (logistic regression) and feature importance (Random Forest) to identify the most influential early predictors.

Text-Based Prediction Models. To evaluate whether PR text alone can predict rejection, we train three text classifiers using PR titles and descriptions concatenated into a single document. As a lightweight linear baseline, we train a **TF-IDF + linear SVM** classifier [56, 57] with class-balanced weighting. We then train a **fastText** supervised classifier [58] as an efficient embedding-based model suitable for short technical text. Finally, we fine-tune a pre-trained **DistilBERT** model [59] for binary sequence classification using standard training settings (max sequence length 128, 3 epochs, learning rate 2×10^{-5}) and incorporate class weights in the loss function to account for imbalance. We compare performance across the structured-feature models and text-only models to assess whether semantic cues in PR text provide a predictive signal beyond lightweight metadata.

LLM-Based Prediction Models. We further evaluate recent general-purpose LLMs as prompting-based baselines for security PR rejection prediction. Specifically, we use **GPT-4o-mini** [60] in both zero-shot and few-shot settings as a larger-model comparison. Unlike the supervised models, these LLM baselines are not fine-tuned on the prediction task. Instead, each model receives only the PR title and description and is asked to predict whether maintainers will merge the PR or reject it by closing it without merging.

For the zero-shot setting, the system prompt describes the model as a GitHub pull-request reviewer assistant and instructs it to return a single JSON object with one of two labels: `{"prediction": "merged"}` or `{"prediction": "rejected"}`. The user prompt contains the PR title and the PR description. To control token cost and reduce prompt-length variation, descriptions are truncated to the first 300 words. For the few-shot setting, we prepend four examples sampled from the training split, with two merged and two rejected PRs. Each example includes the title, a truncated description of up to 100 words, and the corresponding JSON label. The model then predicts the label for a held-out test PR using the same output format.

4. Results

In this section, we present the answers to our research questions.

4.1. RQ1: Prevalence and Distribution of Security-Relevant Agentic PRs

We first examine how frequently autonomous coding agents contribute security-relevant pull requests and how these contributions are distributed across agents. Figure 3 summarizes the prevalence of security-relevant Agentic-PRs. From the curated AIDev dataset, we identified **1,293 security-related PRs**. These 1,293 PRs represent **3.85% of all 33,596 Agentic-PRs** in the curated dataset.

Security-relevant contributions are unevenly distributed across agents. Claude Code shows the highest proportion of security-relevant PRs (**459 PRs – 14.6%**), followed by Copilot (**4,970 PRs – 10%**) and Devin (**4,827 PRs – 7.6%**). OpenAI Codex is the agent with the smallest proportion of security-focused PRs, with only **1.3%** of its PRs confirmed as security-relevant after manual validation.

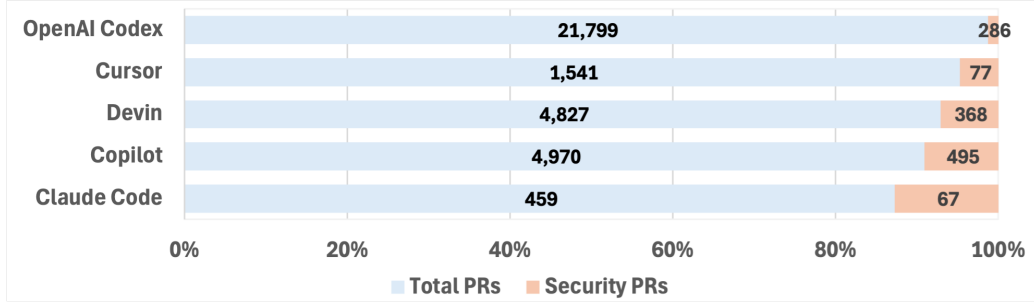


Figure 3: Prevalence of Security-relevant PRs Across Different Autonomous Coding Agents

Distribution Across Projects. The dataset spans **2,807 unique repositories**, yet the distribution of Agentic-PRs is highly skewed (Gini coefficient = 0.83): just **22 repositories** account for 50% of all PRs, and 306 repositories account for 80%. The median repository contributes only **2 PRs**, while the most active single repository (*mochilang/mochi*) contributes 8,913. Security PRs are distributed more evenly across **520 repositories**, with 61.7% of those repositories contributing a single security PR, and only 23 contributing ten or more.

This concentration is agent-specific: **OpenAI Codex** covers the broadest set of repositories (1,248 unique repos, avg. 17.5 PRs/repo), reflecting its pipeline-style bulk operation, while **Claude Code** shows the highest repository diversity relative to its volume (213 repos, avg. 2.2 PRs/repo), suggesting more targeted, one-off engagements.

Longitudinal Stability. The dataset spans **2024-Q4 through 2025-Q3**, with the large majority of PRs concentrated in Q2–Q3 2025 as adoption accelerated (Q1: 1,695 PRs; Q2: 18,697; Q3: 13,139). Figure 4 plots the monthly security-PR rate for confirmed security PRs.

The security-PR rate is broadly stable at **3.8–5.8%** across most months, with an isolated spike to 11.4% in March 2025 driven by a concentrated wave of **Devin** activity on security-sensitive repositories. Excluding that month, the rate remains in the 3.8–6.2% band, suggesting that agents’ propensity to generate security-relevant PRs has not materially changed as adoption grew.

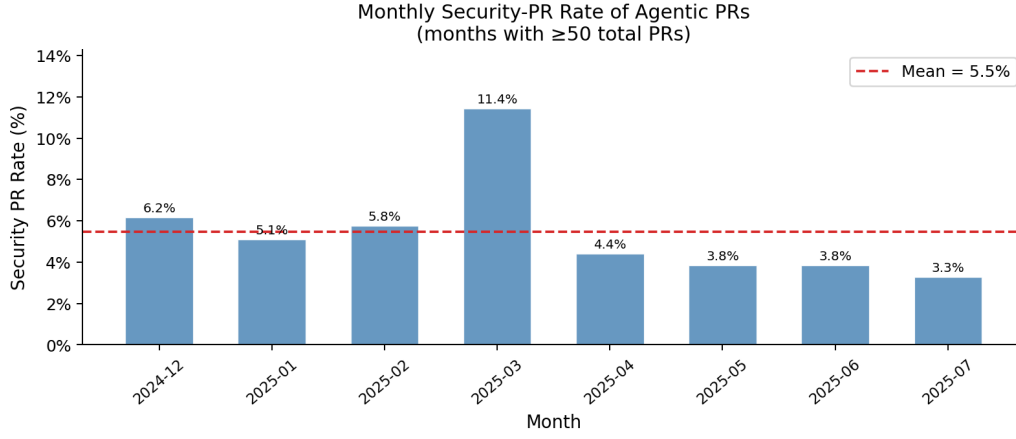


Figure 4: Monthly Security PR Rate of Agentic PRs.

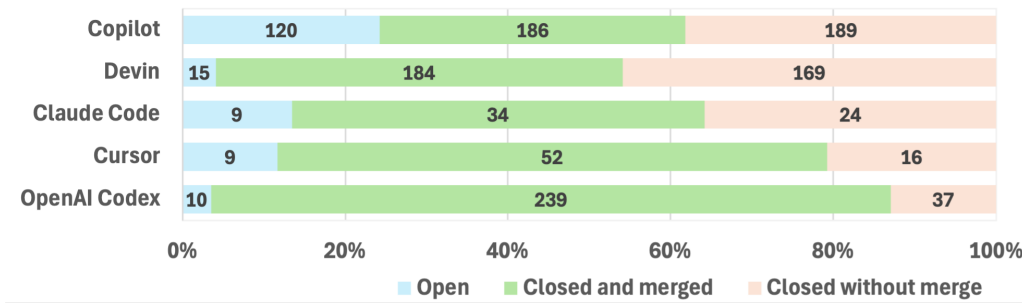


Figure 5: RQ2 Results – Outcomes of Security-Related Pull Requests Authored by Agents.

4.2. RQ2: Security Outcomes Across Agents, Ecosystems, and Code Types

This RQ examines whether the merge rates of security-related PRs vary across autonomous coding agents, programming ecosystems, and types of code changes. We compute the merge rate as the *number of merged PRs* divided by the *total number of closed PRs* (merged or unmerged).

Merge Rates Per Agent. Figure 5 summarizes the statuses of the security-related pull requests authored by each agent. As observed in this figure, we found that **OpenAI Codex** achieves the highest merge rate (86.59%) followed by **Cursor** (76.47%), **Claude Code** (58.62%), and **Devin** (52.12%) while **Copilot** (49.60%) demonstrates the lowest merge rate.

Merge Rates Across Programming Languages. Table 2 reports the most frequent programming languages among security-related Agentic-PRs and their corresponding merge rates, sorted by merge rate. While **TypeScript** accounts for the largest number of security PRs (447 PRs), it exhibits a comparatively lower merge rate (56.51%), suggesting stricter review or higher rejection likelihood in this ecosystem. In contrast, **Python** security PRs (252 PRs) achieve a higher merge rate (68.30%), indicating more favorable acceptance outcomes.

Languages with smaller numbers, such as **Ruby** and **HTML**, exhibit the highest merge rates (above 80%); however, these results should be interpreted cautiously due to limited sample sizes. Mid-volume systems-oriented languages, including **Go**, **Java**, and **C#**, show relatively consistent merge rates around 60–62%, suggesting more uniform reviewer expectations in these ecosystems. Notably, **Rust** security PRs have the lowest merge rate (51.16%), which may reflect heightened scrutiny in safety-critical or performance-sensitive contexts.

Table 2: Top 10 languages for security PRs and their merge rates.

Language	Total # PRs	Merge Rate (%)
Ruby	20	83.33
HTML	26	82.61
Python	252	68.30
JavaScript	48	65.00
Dart	19	63.16
Go	108	61.70
Java	48	60.00
C#	103	60.53
TypeScript	447	56.51
Rust	51	51.16

Merge Rates Per Domain. Table 3 aggregates security-related PRs into broader ecosystem domains. At the domain level, **Data/ML** exhibits the highest merge rate among the major domains (63.71%), followed closely by **Web** (62.11%) and **Enterprise** (60.18%), while **Systems** has the lowest merge rate (56.15%). **Operations** security PRs achieve the highest overall merge rate (69.70%); however, it is important to highlight that this result is based on a very small number of PRs (35) and should therefore be interpreted with caution.

Table 3: Security PR outcomes aggregated by ecosystem domain.

Domain	Total # PRs	Merge Rate (%)
Web	606	62.11
Data/ML	262	63.71
Systems	216	56.15
Enterprise	156	60.18
Operations	35	69.70

Distribution and Merge Rates of Security Change Types. Table 4 summarizes how autonomous coding agents distribute their security-related work across major security change categories and also reports aggregate category totals and merge rates. Overall, *Security Feature* and *Vulnerability Fix* PRs account for the largest share of security-related changes, with *Security Feature* PRs being both the most prevalent and exhibiting higher aggregate merge rates than *Vulnerability Fix* PRs. In contrast, *Configuration/Compliance* and *Dependency Update* PRs occur less frequently but achieve comparatively high merge rates.

At the agent level, variation emerges in how security work is distributed. For example, Claude Code predominantly contributes *Security Feature* PRs (68.7%), whereas Copilot places greater emphasis on *Vulnerability Fix* PRs (50.5%). Cursor and OpenAI Codex exhibit more balanced distributions between these two categories while also contributing a non-trivial share of *Configuration/Compliance* PRs. Overall, these patterns indicate that agents prioritize distinct forms of security work rather than converging on a uniform security contribution profile.

Table 4: Nature of security work by agent: percentage breakdown of categories within each agent’s security PRs. Aggregate category volumes and merge rates are shown at the top.

Agent / Aggregate	Sec. Feat.	Vuln. Fix	Conf./Compl.	Depn. Update
Total # PRs	554	506	134	113
Merge Rate (%)	64.61	53.94	70.08	65.66
Claude Code	68.7	25.4	0.0	6.0
Copilot	33.1	50.5	3.8	12.5
Cursor	40.3	41.6	14.3	3.9
Devin	51.6	27.5	13.9	7.1
OpenAI Codex	40.6	35.7	17.8	5.9

Table 5: Per-agent characteristics for security PRs, ordered by security PR rate (descending). Security rate = confirmed security PRs / all closed PRs for that agent. Issue-routed = security PRs linked to a pre-existing issue. Domain percentages are computed over confirmed security PRs. Gap = non-security minus security merge rate (pp).

Agent	PRs	Sec. %	Merge %		Gap	Issue %	Top domain
			Sec	Non			
Claude Code	459	14.6	58.6	73.6	+15.0	17.9	Web (43.3)
Copilot	4,970	10.0	49.6	55.5	+5.9	81.2	Web (34.7)
Devin	4,827	7.6	52.1	55.8	+3.7	6.8	Web (75.8)
Cursor	1,541	5.0	76.5	74.5	-2.0	9.1	Web (61.0)
Codex	21,799	1.3	86.6	85.8	-0.8	2.4	Data/ML (38.1)

Agent-level variation. Table 5 shows that security engagement varies widely across agents: security PRs account for only 1.3% of OpenAI Codex’s closed PRs but 14.6% of Claude Code’s. Three patterns help explain this variation.

Integration mode. Table 5 shows that Copilot is the most issue-driven agent: 81.2% of its security PRs are linked to a pre-existing issue. This suggests that Copilot often responds to human-scoped security concerns rather than selecting security tasks independently. In contrast, OpenAI Codex and Cursor have much lower issue-routing rates, 2.4% and 9.1%, respectively, indicating a more self-directed mode of operation. This difference is also reflected in outcomes: Codex has the highest security merge rate (86.6%) and almost no security penalty relative to non-security PRs (−0.8 pp), consistent with narrower, more selective fixes.

Project domain. The dominant domain also differs by agent. Codex’s security PRs are most often in Data/ML repositories (38.1%), whereas Devin, Cursor, Claude Code, and Copilot are led by Web projects, with Web accounting for 75.8%, 61.0%, 43.3%, and 34.7% of their security PRs, respectively. This distribution suggests that agents are not engaging with the same security workload: some operate mainly in Web settings, where security fixes may involve authentication, input handling, or request-flow context, while Codex’s activity is more concentrated in Data/ML repositories.

Security breadth versus acceptance. The security rate and merge rate capture different aspects of agent behavior. Claude Code has the highest security-PR rate (14.6%) but also the largest merge-rate gap: its security PRs merge at 58.6%, compared with 73.6% for non-security PRs (+15.0 pp).

This suggests broader security engagement but lower acceptance once those changes reach review. Codex shows the opposite pattern: it has the lowest security-PR rate (1.3%) but the highest security merge rate (86.6%). Thus, security engagement and acceptance are not interchangeable measures. The former reflects how often an agent attempts security work, whereas the latter reflects how readily those attempts are accepted by maintainers.

Longitudinal Stability for Merge Rate. The quarterly merge rate for security PRs rises from 42.2% in Q1 2025 to **66.3%** in Q2 and stabilizes at 60.8% in Q3, consistent with two confounding factors: (i) Devin dominates Q1 (133 of 135 closed security PRs) and exhibits one of the lower per-agent merge rates (42.4% in Q1), naturally depressing the aggregate; and (ii) Q2 onward introduces Copilot and OpenAI Codex at scale, both with distinct merge dynamics (Codex: 84.7% $\rightarrow$ 92.5%; Copilot: 48.5% $\rightarrow$ 50.9%).

4.3. RQ3 Results: Security Actions and Intents in Agentic-PRs

After open coding 1,293 manually confirmed security-relevant Agentic-PRs from the curated AIDev subset, we identify recurring *security actions* (what agents do) and *security intents* (why agents do it). We report only themes that appear in at least 10 PRs to focus on stable and recurrent patterns.

4.3.1. Security Actions Performed by Autonomous Agents

Distribution of Security Actions and Intents. Table 6 summarizes the most frequent *security actions* and *security intents* identified through open coding across the 1,293 security-relevant Agentic-PRs. Actions capture *how* agents modify the codebase, while intents capture *why* those changes are made.

Overall, agentic security work is dominated by a small number of recurring implementation patterns. On the action side, *Code Refactoring* (957), *Testing* (755), and *Documentation* (692) are most prevalent, indicating that security improvements are often realized through restructuring existing code, adding or extending tests, and clarifying usage or constraints rather than introducing entirely new mechanisms. Lower-level but still common actions such as *Error Handling*, *Configuration Management*, and *Input Validation* further suggest a focus on robustness and defensive hardening.

On the intent side, *Functionality Improvement* (890) and *Vulnerability Mitigation* (741) are the two dominant motivations. Notably, a large fraction

Table 6: Security actions and intents identified via open coding (themes with fewer than 10 occurrences omitted).

Actions			
Theme	#	Theme	#
Code Refactoring	957	Testing	755
Documentation	692	Error Handling	595
Security Improvements	580	API Development	540
Configuration Management	495	User Experience Enhancement	484
Input Validation	434	Authentication & Authorization	377
Dependency Management	367	Performance Optimization	313
Logging and Monitoring	262	Compatibility Maintenance	179
Version Control	143	Database Management	49
Others	28		

Intents			
Theme	#	Theme	#
Functionality Improvement	890	Vulnerability Mitigation	741
User Experience	697	Error Handling	604
Security Enhancement	531	Testing and Reliability	468
Compatibility Assurance	457	Performance Optimization	389
Code Quality	376	User Guidance	340
Maintainability	335	Compliance and Standards	255
Documentation Improvement	218	Development Efficiency	183
Resource Management	82		

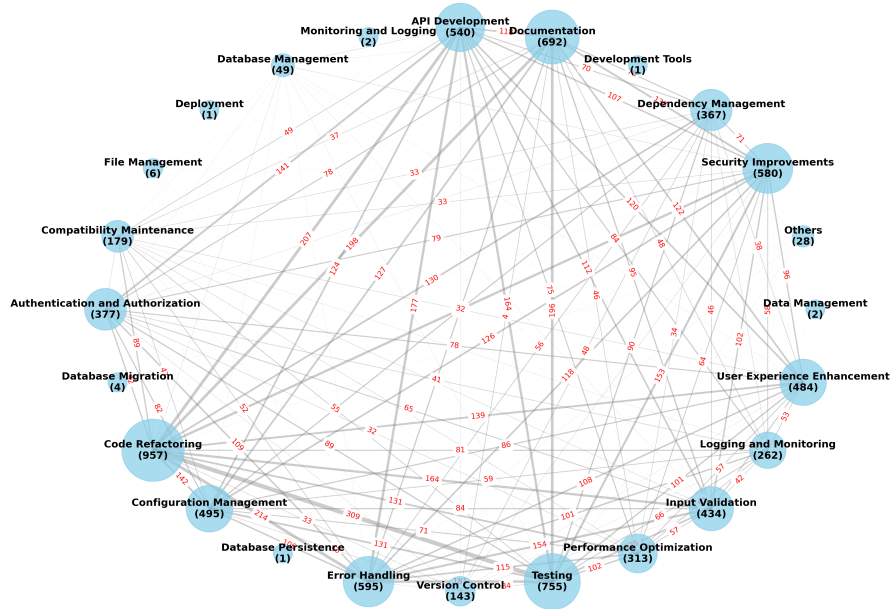


Figure 6: Co-occurrence Network of Security Actions.

of security-relevant PRs are framed as improving functionality or developer experience rather than narrowly fixing a specific vulnerability, reinforcing that security work in practice is often intertwined with broader maintenance and evolution goals. Intents related to *User Experience*, *Reliability*, and *Compatibility Assurance* also appear frequently, reflecting a preventive and quality-oriented framing of security changes.

Action–Intent Co-occurrence Patterns. To move beyond marginal frequencies, we analyze how security-related actions and intents co-occur within the same Agentic-PRs. We quantify these associations using pairwise *lift*, which measures how much more frequently two themes co-occur than would be expected if they were independent, and the ϕ coefficient, which captures the strength of correlation between two binary-coded themes. We identify recurring and statistically strong links between *how* agents modify code and *why* those changes are made. The resulting co-occurrence patterns show that agentic security work is rarely one-dimensional. Instead, agents systematically pair concrete implementation techniques with aligned security motivations. In Figure 6 and Figure 7, we present their co-occurrence networks. To complement the co-occurrence network visualization and improve readability, Table 7 summarizes representative high-strength co-occurrence links identified in the RQ3 analysis.

Table 7: Strong co-occurrence links from the RQ3 action–intent analysis.

Theme 1	Theme 2	Lift	ϕ
Documentation	User Guidance	2.77	0.41
Documentation	Documentation Improvement	2.70	0.31
User Experience Enhancement	User Experience	2.50	0.43
Performance Optimization	Performance Optimization	3.50	0.39
Performance Optimization	Resource Management	3.54	0.17
Error Handling	Error Handling	2.05	0.31
Dependency Management	Version Control	2.38	–
API Development	Database Management	2.46	–

From the results, we found several strong and intuitive associations. *Documentation* actions exhibit one of the strongest links to intent, co-occurring frequently with both *User Guidance* (lift = 2.77, ϕ = 0.41) and *Documentation Improvement* (lift = 2.70, ϕ = 0.31). This indicates that documentation-related security PRs are commonly motivated by steering correct and secure usage rather than by purely descriptive goals. Similarly, *User Experience Enhancement* actions show a strong association with *User Experience* intents

(lift = 2.50, ϕ = 0.43), suggesting that agents often frame security improvements as usability fixes that reduce misuse or error-prone interactions.

Performance-related work displays particularly strong coupling. *Performance Optimization* actions co-occur most frequently with both *Performance Optimization* intents (lift = 3.50, ϕ = 0.39) and *Resource Management* intents (lift = 3.54, ϕ = 0.17). These high-lift associations suggest that performance tuning is often treated as security-relevant when it mitigates resource exhaustion, reliability issues, or denial-of-service risks. Likewise, *Error Handling* actions strongly align with *Error Handling* intents (lift = 2.05, ϕ = 0.31), reflecting direct remediation of failure modes that could otherwise be exploited.

Beyond direct action–intent pairs, we also observe meaningful co-occurrence among actions themselves. For example, *Dependency Management* frequently co-occurs with *Version Control* actions (lift = 2.38), and *API Development* co-occurs with *Database Management* (lift = 2.46), indicating that security-related changes often span multiple technical layers within a single PR. Overall, these co-occurrence patterns demonstrate that agentic security PRs are not collections of isolated fixes. Instead, autonomous agents consistently align specific implementation choices with coherent security rationales. This reinforces the analytical value of separating *actions* from *intents* and shows that agent-authored security work blends vulnerability remediation, preventive hardening, and quality improvement within broader software engineering activities.

Intent–Action Alignment. To assess whether intent and action labels co-occur coherently, we classify each PR into one of four alignment quadrants based on whether its open-coding labels include at least one security-oriented intent (*Security Enhancement*, *Vulnerability Mitigation*, *Compliance and Standards*) and at least one security-oriented action (*Security Improvements*, *Authentication and Authorization*, *Input Validation*, *Logging and Monitoring*).

Notably, **intent–action misalignment does not predict rejection**. Merge rates across the four quadrants are nearly identical: fully aligned PRs (security intent + security action, n = 743) merge at **62.6%**, PRs with a security intent but general action (n = 342) at **63.5%**, PRs with a general intent but security action (n = 388) at **62.6%**, and PRs with neither (n = 777) at **62.5%**. This uniformity suggests that reviewers respond to the *presence* of

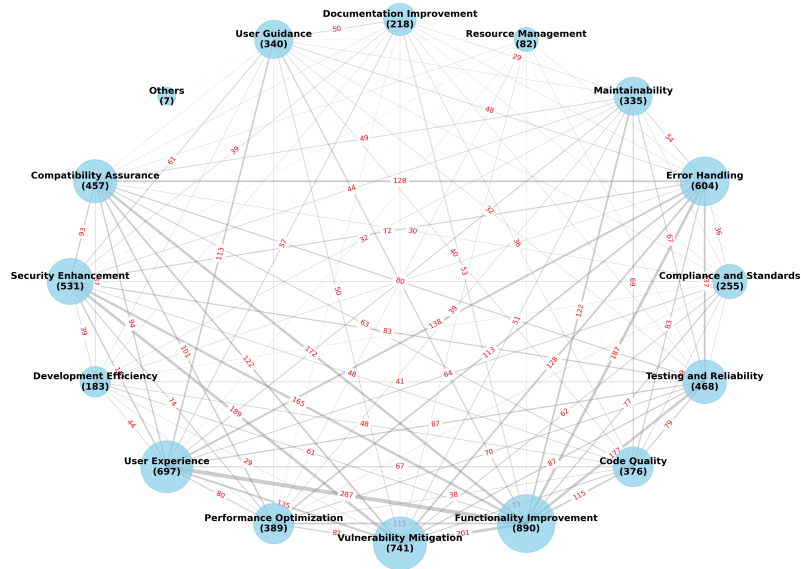


Figure 7: Co-occurrence Network of Security Intents.

security-sensitive code changes, not to whether the agent’s stated motivation is security-framed.

The most common misalignment pattern is *Authentication and Authorization* actions paired with *Functionality Improvement* intent: agents frequently introduce or extend authentication mechanisms as part of a broader feature addition, without framing the PR as a security change. Similarly, *Input Validation* frequently appears alongside *Development Efficiency* or *Code Quality* intents, reflecting security hardening bundled into general refactoring work. These cases illustrate that agent-authored security work is often *embedded* in multi-purpose PRs rather than isolated as dedicated security fixes.

Non-Security Intents in Security PRs. Two intents, *Performance Optimization* (1,242 co-occurrences) and *Development Efficiency* (663), appear frequently despite seeming only tangentially related to security. Both arise because agent-authored security PRs are rarely single-purpose: the most common actions co-occurring with *Performance Optimization* intent are *Code Refactoring* (13%), *Error Handling* (10%), and *Testing* (9%), with *Security Improvements* accounting for 7%. Among confirmed security PRs specifically, *Performance Optimization* appears in 157 cases, primarily when fixing a vulnerable code path simultaneously removes an inefficiency (e.g., replacing a vulnerable regex with a hardened, faster implementation). *Development Efficiency* co-occurs most with *Documentation* (14%) and *Testing* (11%), reflecting agents that add security-related tests or update documentation while addressing a vulnerability. These intents therefore capture real bundling behavior in agent-authored PRs: agents embed security changes within broader improvement tasks rather than submitting focused, single-intent patches.

4.4. RQ4: Reviewer Behavior Toward Security Agentic-PRs

To understand whether human reviewers apply different levels of scrutiny to security-relevant Agentic-PRs compared to non-security Agentic-PRs. We report results along three dimensions: (i) aggregate review outcomes and latency, (ii) interaction-based scrutiny signals, and (iii) agent- and category-level heterogeneity.

Aggregate Review Outcomes and Latency. Table 8 summarizes review latency statistics. From the results, we found that security-relevant Agentic-PRs have consistently lower merge rates than non-security PRs; i.e., security PRs

are merged at only **61.50%**, compared to **77.33%** for non-security PRs. We also found that Security PRs have a median review latency of **3.92** hours, compared to only **0.11** hours for non-security PRs, and a mean latency more than twice as long (97.45 vs. 38.29 hours). This difference is statistically significant ($p < 0.001$), indicating substantially increased review time for confirmed security-relevant Agentic-PRs.

Table 8: Review latency (hours) for security vs. non-security merged Agentic-PRs.

PR Type	Total # PRs	Mean	Median	Std. Dev.
Security	1,130	97.45	3.92	239.91
Non-Security	30,154	38.29	0.11	140.42

Interaction-Based Scrutiny Signals. Beyond time and acceptance outcomes, we examine direct signals of reviewer engagement. Security Agentic-PRs receive **more than twice as many review comments** on average (2.5 vs. 1.0) and trigger **over twice as many formal review events** (1.8 vs. 0.8), with statistically significant differences across all interaction types ($p < 0.001$). Notably, security PRs are more likely to receive explicit *Changes Requested* actions (0.1 vs. 0.0) and approvals accompanied by discussion, indicating active reviewer intervention rather than passive acknowledgment. Together, these interaction patterns indicate that increased latency is not merely idle waiting time, but corresponds to deeper reviewer engagement, discussion, and corrective feedback when agents modify security-sensitive code.

Agent-Level Heterogeneity. Agent-level analysis (Table 9) shows that this effect holds across all agents, though with heterogeneous magnitude. Security-related PRs authored by **Devin** experience the largest median delay (+16.08 hours), followed by **Copilot** (+5.46 hours) and **Claude Code** (+3.42 hours), while **OpenAI Codex** exhibits the smallest observed median latency difference in our dataset.

To better understand this unusually low aggregate latency for Codex, we further examine the role of PR scope using the size-stratified analysis and agent-level description-length comparisons. These analyses indicate that review outcomes are strongly associated with PR size, and that Codex security PRs in our dataset tend to have shorter descriptions than those of other agents, suggesting a narrower PR scope. Accordingly, the aggregate Codex latency

should be interpreted cautiously as a property of the observed Codex security PR corpus in our dataset rather than as evidence that Codex-authored security PRs are inherently reviewed faster across projects.

Table 9: Agent-specific review latency (hours) for security vs. non-security PRs.

Agent	Sec. Med.	Sec. Std.	Non-Sec. Med.	Non-Sec. Std.	Delta
Devin	23.76	312.60	7.68	173.18	+16.08
Copilot	18.32	241.49	12.87	215.95	+5.46
Claude Code	5.13	139.22	1.71	213.15	+3.42
Cursor	1.08	93.23	0.89	95.41	+0.18
OpenAI Codex	0.07	125.87	0.02	109.16	+0.05

Merge rates further reinforce this pattern. While Codex security PRs are merged at a high rate (86.6%), Copilot and Devin security PRs are merged in only about half of cases. These differences suggest that reviewer scrutiny is shaped not only by security relevance but also by prior experience and perceived reliability of specific agents.

Security Action Categories. To move beyond aggregate counts, we further examine how review latency varies across different *security action categories*. Table 10 summarizes review latency statistics conditioned on the primary type of security change.

Table 10: Review latency (hours) for security-related Agentic-PRs by security change category.

Category	Total # PRs	Mean	Std. Dev.	Median
Config / Compliance	127	127.17	341.67	0.86
Security Feature	486	105.53	253.86	9.95
Vulnerability Fix	432	84.19	190.43	3.35
Dependency Update	99	83.40	185.83	4.36

Security Feature PRs exhibit the longest median review latency (**9.95** hours), substantially exceeding that of Vulnerability Fix PRs (**3.35** hours) and Dependency Update PRs (**4.36** hours). In contrast, Config/Compliance PRs are reviewed markedly faster, with a median latency of only **0.86** hours. These differences suggest that reviewers distinguish between classes of security work: changes that introduce or extend security mechanisms tend to trigger deeper inspection and longer deliberation, whereas configuration-level or compliance-oriented changes are often resolved more quickly.

Table 11: Merge rate by PR size (description-length tertile) for closed security and non-security Agentic-PRs. Small: <36 words; Medium: 36–69 words; Large: >69 words. The aggregate security–non-security gap vanishes for small and medium PRs and is substantially reduced for large ones.

Size	Security PRs		Non-Security PRs		Gap (pp)	p
	n	MR (%)	n	MR (%)		
Small	125	86.4	10,175	84.6	−1.8	0.67
Medium	153	86.3	11,220	84.1	−2.2	0.53
Large	852	53.4	8,759	60.2	+6.8	<0.001*
Overall	1,130	61.5	30,154	77.3	+15.8	<0.001*

Role of PR Complexity. Finally, we assess whether heightened scrutiny can be explained solely by PR size or complexity. Security Agentic-PRs are indeed larger and more complex, touching nearly **twice as many files** (30.3 vs. 15.4), exhibiting **higher code churn** (5,301 vs. 1,556 lines), and containing **longer descriptions** (256.6 vs. 97.2 words), with all differences statistically significant.

To assess whether heightened scrutiny persists *after controlling for size*, we stratify all closed PRs into tertiles based on description length (small: <36 words; medium: 36–69 words; large: >69 words) and compare merge rates within each stratum (Table 11).

The size-stratified analysis reveals that **the aggregate security–non-security merge gap is largely driven by PR size**. For small and medium PRs, security and non-security merge rates are statistically indistinguishable (small: 86.4% vs. 84.6%, $p = 0.67$; medium: 86.3% vs. 84.1%, $p = 0.53$). The gap emerges only among large PRs (53.4% vs. 60.2%, $p < 0.001$), where security PRs still merge **6.8 percentage points** less often, suggesting a residual scrutiny effect for complex security changes. However, because the majority of security PRs fall in the large stratum ($n = 852$ of 1,130 closed), the aggregate 15.8 pp gap reflects primarily a size confound rather than a uniform security-content penalty: same-sized security and non-security PRs are treated with comparable acceptance rates. Time-to-close mirrors this pattern: among large PRs, security PRs take a median **16.5 hours** to close vs. **5.7 hours** for non-security PRs of equivalent size, while small and medium PRs close within minutes regardless of security relevance.

This complexity gap is further pronounced at the agent level. Using descrip-

tion length as a within-dataset proxy, we find a $10\times$ spread: OpenAI Codex security PRs have a median description of 42 words, versus 336 words for Copilot and 421 words for Claude Code. Critically, this difference persists *within each semantic category*: for Vulnerability Fix PRs alone, the median body length is 37 words for Codex versus 356 words for Copilot and 477 words for Claude Code, while Codex’s merge rate (84%) and median latency (<6 minutes) remain uniform across all four categories. These patterns suggest that the current four-category scheme captures *intent* but not *scope*: agents with lower contribution rates tend to handle simpler variants of each security type, and a finer-grained complexity dimension, such as lines changed or number of files affected, would be needed to isolate the contribution of reviewer scrutiny from task self-selection in future work.

However, the persistence of higher review latency, lower merge rates, and increased reviewer interaction, both across agents and within comparable security categories, suggests that reviewers respond not only to PR scale, but also to the perceived risk and responsibility associated with agent-authored security changes.

Comparing with Human-authored Security PRs. We further compare security Agentic-PRs with **634** manually confirmed human-authored security PRs from the same repositories, and additionally examine how human reviewers treat security PRs relative to non-security PRs authored by humans. Among human-authored PRs, security PRs are accepted at a rate of **79.2%**, compared to **83.5%** for non-security PRs, a **4.3 percentage-point** gap that is statistically significant ($p = 0.011$). While human-authored security PRs also take longer to close (median **7.00** vs. **4.41** hours, $p < 0.001$), the absolute reduction in acceptance is modest.

This establishes an important baseline: security content causes a small but measurable dip in acceptance rates even for human authors, reflecting the inherently higher stakes of security-sensitive changes. In contrast, agentic-authored security PRs are merged at only **61.5%**, a **17.7 percentage-point (pp)** gap relative to human security PRs ($p < 0.001$). The agent security penalty, measured as the gap between non-security and security merge rates within each author type, is **15.8 pp** for agents versus only **4.3 pp** for humans, making the agent-specific security scrutiny effect **3.7 $\times$** larger. Furthermore, even for non-security PRs, agents are merged less often than humans (**77.3%** vs. **83.5%**, $\Delta = 6.2$ pp, $p < 0.001$), confirming a general reviewer wariness

toward automated contributions. The amplification of this wariness for security content, from 6.2 pp to 17.7 pp, indicates that reviewers apply more heightened scrutiny to agent-authored security changes beyond any baseline distrust of automation.

4.5. RQ5: Early Predictors of Risky or Rejected Security Agentic-PRs

This research question examines whether early, observable signals available at pull-request creation time can help predict whether a security-relevant Agentic-PR will be rejected (*i.e.*, closed without merge). Our analysis focuses on the *curated* dataset of manually validated security-related Agentic-PRs. In total, we analyze **1,130** closed security PRs, of which **435** are rejected, corresponding to a rejection rate of **38.5%**.

Table 12: Association between early signals and PR rejection in the curated security PR dataset (logistic regression coefficients).

Feature	Coefficient (β)
Title length	0.316
PR size	0.049
Description length	0.049
Sensitive keyword in title	-0.054
Agent (encoded)	-0.268

Among the examined early signals, title length shows the strongest positive association with PR rejection, exhibiting the largest logistic regression coefficient ($\beta = 0.316$), followed by PR size and description length (both $\beta \approx 0.049$). Consistent with these associations, rejected PRs tend to be larger and more verbose on average: rejected PRs involve a mean of 2,720.3 lines changed compared to 1,873.1 for merged PRs, and have longer titles on average (59.8 vs. 52.7 characters). Together, these results suggest that early signals related to PR complexity and explanatory burden are associated with a higher likelihood of rejection. In contrast, the presence of sensitive security-related keywords in PR titles exhibits a weak negative association with rejection ($\beta = -0.054$) and low importance across predictive models. This indicates that explicitly referencing security-sensitive topics (*e.g.*, authentication, cryptography, credentials) is not, by itself, a strong early signal of rejection for security-related Agentic PRs.

We next evaluate the predictive power of early signals using structured-feature, text-based models, and prompting-based LLM baselines (Table 13).

Table 13: Predictive performance for rejection prediction on the curated security-related Agentic-PR dataset (held-out test set, $n = 339$, 38.3% rejected). Metrics are reported for the *Rejected* class.

Model	Accuracy	Rej. Precision	Rej. Recall	Rej. F1
<i>Structured-feature models</i>				
Logistic Regression	0.58	0.46	0.48	0.47
Random Forest	0.61	0.49	0.41	0.45
<i>Text-based models (trained)</i>				
SVM (TF-IDF)	0.65	0.53	0.72	0.61
fastText	0.71	0.62	0.60	0.61
DistilBERT (fine-tuned)	0.64	0.52	0.85	0.64
<i>LLM baselines</i>				
GPT-4o-mini (zero-shot)	0.60	0.45	0.18	0.26
GPT-4o-mini (4-shot)	0.54	0.34	0.20	0.25

Among structured-feature approaches, logistic regression and Random Forest achieve modest performance, with accuracies of 0.58 and 0.61, respectively, and rejected-class F1-scores below 0.50. Feature-importance analysis for the Random Forest indicates that PR size, description length, and title length dominate prediction, whereas agent identity and the presence of sensitive security-related keywords contribute comparatively little.

Text-based models outperform structured-feature approaches in identifying rejected PRs. A TF-IDF + linear SVM classifier achieves a rejected-class F1-score of 0.61, while a fastText classifier attains comparable rejected-class performance ($F1 = 0.61$) with the highest overall accuracy (0.71). Fine-tuning a DistilBERT model further increases rejected-class recall to 0.85 and achieves the highest rejected-class F1-score (0.64), indicating that semantic cues in PR titles and descriptions provide a strong predictive signal for rejection.

In contrast, the prompting-based LLM baselines perform poorly on rejection detection. GPT-4o-mini in the zero-shot setting reaches 0.60 accuracy, but its rejected-class recall is only 0.18, yielding a rejected-class F1-score of 0.26. Adding four in-context examples does not improve performance; the 4-shot setting decreases accuracy to 0.54 and produces a similar rejected-class F1-score of 0.25. This suggests that, without task-specific fine-tuning or calibration, GPT-4o-mini tends to under-identify rejected PRs and is less effective than trained text classifiers for this prediction task.

Overall, these results indicate that PR rejection is difficult to predict from

metadata and PR text alone. Although trained models achieve modest macro-F1 scores of 0.58–0.60, this performance reflects a low signal ceiling rather than a failure of a particular modeling approach. Rejection depends heavily on reviewer judgment and contextual factors that are not observable from the PR itself, including reviewer expertise, repository norms, project history, and interpersonal trust between the agent operator and maintainers. Text-based models still provide stronger signals than lightweight structured metadata, suggesting that rejection risk is partially encoded in the semantic content and verbosity of PR descriptions. However, the weak performance of GPT-4o-mini under zero-shot and four-shot prompting despite access to the full PR text confirms that general-purpose LLM reasoning alone is insufficient for reliable rejection prediction in this setting.

5. Discussion

In this section, we synthesize our findings and discuss their implications for agent security participation, human review dynamics, and the design of autonomous coding agents in security-sensitive software engineering workflows.

5.1. Security Work Is a Meaningful but Secondary Part of Agent Activity

Our results show that security-related Agentic-PRs constitute a *non-trivial but minority* portion of agent-authored contributions. In the curated AIDev dataset, only **3.85%** of Agentic-PRs were manually confirmed as security-relevant (RQ1). While this fraction is small relative to total agent activity, it nevertheless corresponds to over a thousand security-related changes, indicating that autonomous agents are already engaging with security concerns at scale in real-world repositories.

Importantly, this security work is not limited to narrow vulnerability patching. Our qualitative analysis (RQ3) reveals that agents frequently perform substantive security-related actions such as refactoring security-sensitive code, improving error handling, strengthening authentication or input validation logic, managing dependencies, and enhancing tests and documentation. However, many of these actions are framed around broader software engineering goals, such as functionality improvement, usability, maintainability, or compatibility, rather than explicitly as vulnerability remediation. This suggests that agentic security work often takes the form of *preventive hardening*

and quality improvement embedded within routine development tasks, rather than isolated “security-only” interventions.

At the same time, security-related outcomes vary substantially across agents, ecosystems, and change types (RQ2). Some agents consistently achieve high merge rates for security PRs, while others experience lower acceptance, even within the same curated dataset. Similarly, security feature PRs and refactoring-oriented changes tend to be treated differently from vulnerability fixes or dependency updates. These patterns indicate that agent effectiveness in security contexts depends not only on whether a change is security-related, but also on *how that security work is expressed, scoped, and integrated* into existing development practices.

5.2. Security Actions and Intents Reveal Structured Agent Behavior

By separating *security actions* (what agents do) from *security intents* (why they do it), our open-coding analysis exposes systematic patterns in agent-authored security work (RQ3). On the action side, a small set of recurring behaviors, such as code refactoring, testing, documentation, error handling, and configuration management, dominates agentic security contributions. On the intent side, functionality improvement and vulnerability mitigation emerge as the most common motivations, followed closely by user experience, reliability, and compatibility concerns.

Crucially, co-occurrence analysis shows that these actions and intents are not randomly paired. Instead, agents consistently align specific implementation strategies with coherent motivations. For example, documentation actions are strongly associated with user guidance and documentation-improvement intents, while performance optimizations align closely with both performance and resource-management intents. Error-handling actions frequently co-occur with error-handling intents, reflecting direct remediation of failure modes that could otherwise be exploited. These structured action–intent pairings indicate that agentic security PRs are not ad hoc collections of fixes, but rather reflect internally consistent reasoning about security-related changes.

This finding reinforces the analytical value of distinguishing actions from intents: it allows us to capture how agents operationalize security goals through concrete code changes, and how security is often framed as a byproduct of improving robustness, usability, or system quality rather than as an isolated

objective.

5.3. Human Reviewers Apply Heightened and Differentiated Scrutiny to Security PRs

Across multiple complementary signals, our results show that human reviewers apply *heightened and differentiated scrutiny* to security-relevant Agentic-PRs compared to non-security Agentic-PRs (**RQ4**). Security PRs are merged significantly less often and remain under review substantially longer. In the curated dataset, only **61.5%** of security Agentic-PRs are merged, compared to **77.3%** of non-security PRs, while median review latency increases by more than an order of magnitude (3.92 vs. 0.11 hours). These differences indicate that reviewers do not treat agent-generated security changes as routine maintenance tasks.

Importantly, increased review latency is accompanied by *greater reviewer engagement*. Security Agentic-PRs receive more than twice as many review comments and formal review events, and are more likely to trigger explicit *Changes Requested* actions. This suggests that longer review times reflect active inspection, discussion, and corrective feedback rather than passive queuing or delayed attention. Taken together, these interaction-based signals indicate that reviewers invest additional cognitive effort when evaluating agent-authored changes that affect security.

Scrutiny is not applied uniformly across agents. Agent-level analysis reveals substantial heterogeneity: security PRs authored by Devin and Copilot experience pronounced delays and lower merge rates, whereas OpenAI Codex security PRs exhibit comparatively low observed review latency in our dataset. However, this pattern should be interpreted cautiously. The size-stratified analysis and agent-level description-length comparisons suggest that differences in PR scope may contribute to the comparatively favorable outcomes observed for Codex. Accordingly, the observed Codex latency pattern should be interpreted as dataset-specific rather than as a universal property of Codex-authored security changes. This pattern suggests that reviewer behavior is shaped not only by security relevance but also by prior experience, familiarity, and perceived reliability of specific agents. Reviewers appear to calibrate their scrutiny based on both *what* is being changed and *who* (or which agent) authored the change.

Reviewers further differentiate scrutiny based on the *type of security work*.

Security feature PRs, which introduce or extend security mechanisms, exhibit the longest review latencies, while vulnerability fixes and dependency updates receive intermediate scrutiny. In contrast, configuration and compliance-related PRs are reviewed markedly faster. This stratification indicates that reviewers assess the semantic risk of security changes and devote deeper inspection to PRs that may alter security guarantees or introduce new attack surfaces.

While security Agentic-PRs are larger and more complex on average—touching more files, exhibiting higher code churn, and containing longer descriptions—these factors alone do not fully explain the observed review behavior. Heightened scrutiny persists across agents and within comparable security categories, suggesting that reviewers respond not merely to PR size, but to the perceived responsibility and potential consequences of agent-authored security changes.

Overall, our findings indicate that human reviewers act as an adaptive risk filter for agent-generated code: they slow down, ask more questions, and apply stricter acceptance criteria when agents modify security-sensitive components. This behavior reflects nuanced human judgment rather than blanket distrust and highlights the continued importance of human oversight in agentic software development pipelines.

5.4. PR Complexity Is More Strongly Associated with Rejection Than Security Content Is

Our analysis of early predictors of rejection (RQ5) further clarifies how reviewers evaluate security-related Agentic-PRs. Rejection is most strongly *associated* with indicators of complexity and explanatory burden: body length ($\Delta_{\text{macro-F1}} = 0.286$), number of structured sections (0.163), and title length (0.148), rather than with explicit references to security topics. Keyword-based security signals contribute modestly (0.060–0.075), and agent identity (0.112) and Vulnerability Fix category (0.117) are more informative than keyword presence alone. These associations are consistent with the size-stratified analysis in RQ4, where the security–non-security merge-rate gap vanishes for small and medium PRs and persists only for large ones.

All trained models achieve only moderate macro-F1 (0.58–0.60), a ceiling shared by GPT-4o in zero-shot evaluation (0.47), confirming that rejection is shaped by factors absent from the PR text itself: reviewer expertise, organiza-

tional norms, and agent-specific trust, rather than by surface features alone. We therefore interpret the feature-importance results as *associations*, not causal drivers: complexity proxies are the most informative observable correlates of rejection, but no causal mechanism is established. Taken together, these findings suggest that reviewers respond to the scope and presentation of agent-authored security changes more than to their security topic, and that acceptance is more likely when changes are narrow, well-documented, and clearly scoped.

5.5. Implications for Designing Review-Aware Secure AI Teammates

Our RQ4 and RQ5 findings have several implications for the design and deployment of autonomous coding agents in security-sensitive workflows. First, our results suggest that review difficulty is associated with the scope and presentation of security Agentic-PRs: security PRs exhibit lower merge rates and longer review latency than non-security PRs (Table 8), while title length, PR size, and description length are the strongest predictors of rejection in RQ5 (Table 12). Second, the additional scrutiny applied to security Agentic-PRs is not uniform: its magnitude varies across agents (Table 9) and across security change categories (Table 10), suggesting that review-aware agent design should be evaluated in a context-sensitive rather than repository-agnostic manner. Third, Table 13 shows that trained text-based models outperform both structured-feature models and prompting-based LLM baselines, suggesting that the semantic content of PR titles and descriptions carries important signals for review outcomes and should therefore be treated as part of the agent output being evaluated.

Overall, our study shows that autonomous coding agents are already participating meaningfully in security-relevant development, and that their review outcomes are associated not only with the underlying code change but also with how that change is scoped, described, and situated within human review processes. Designing AI teammates that are not only technically capable, but also review-aware and context-sensitive is essential as agentic systems become increasingly embedded in collaborative software engineering practice.

5.6. Threats to Validity

As with any large-scale empirical study, our findings are subject to several threats to validity, which we discuss below along with the steps taken to mitigate them.

5.6.1. Construct Validity

A primary threat to construct validity concerns how we operationalize *security relevance*. We identify security-related Agentic-PRs using keyword-based filtering and heuristic rules applied to PR titles and descriptions. While this approach is consistent with prior empirical security studies, it may still yield false positives or false negatives: PRs that do not match our keyword filters are not manually inspected and are therefore treated as non-security-related, even though some may still contain security-relevant changes.

To mitigate this risk, we deliberately designed our keyword set to be *broad and recall-oriented*, covering vulnerability terminology, authentication and authorization concepts, cryptographic and data-protection terms, exploit identifiers, and compliance-related language. This inclusive strategy reduces the likelihood of systematically missing entire classes of security-relevant PRs. Candidate PRs identified through keyword matching were then manually inspected to remove non-security-related cases. Specifically, two authors with 4–6 years of experience in software development and software security research independently reviewed the 2,598 candidate PRs. To assess annotation reliability, we double-coded a statistically representative sample of 335 PRs (95% confidence level, 5% margin of error), yielding a Cohen’s κ score of **0.91**, which indicates *almost perfect agreement* [48]. While PRs that did not match any keyword were not manually reviewed and may still contain security-relevant changes, we expect such cases to be comparatively rare given the breadth of the keyword set and the subsequent manual validation step.

Another limitation is that our keyword filter operates on PR titles and descriptions, but not on the content of linked GitHub issues. Of the 1,293 confirmed security PRs, **35%** (452 PRs) have at least one linked issue, and **46%** of those linked issues contain security-relevant keywords in their own title or body. Because all issue references in our dataset are embedded in PR descriptions (rather than inferred from repository metadata), PRs that explicitly link a security issue tend also to describe the security concern in the PR body.

A second construct validity concern arises from using merge status and review latency as proxies for security outcomes. These measures capture *perceived risk and reviewer scrutiny* rather than ground-truth security correctness. Accordingly, our conclusions focus on how security-related Agentic-PRs are

treated during human review, not on whether merged PRs are objectively secure or rejected PRs are insecure. We further mitigate this threat by complementing quantitative analyses with qualitative open coding (RQ3), which provides insight into the nature and intent of security-related changes beyond outcome metrics alone.

5.6.2. Internal Validity

Threats to internal validity stem from confounding factors such as repository-specific contribution norms, CI failures, reviewer availability, or concurrent project activity that may influence merge decisions and review timelines. While it is infeasible to control for all such factors at scale, we mitigate this threat by comparing security and non-security PRs *within the same repositories* and by analyzing a manually curated subset of popular repositories.

Observed differences across agents may also reflect unmeasured factors such as deployment context, prompting strategies, task allocation, or configuration settings. Consequently, our findings should be interpreted as *descriptive associations rather than causal effects*.

5.6.3. External Validity

Our study is based on the AIDev dataset, which captures Agentic-PRs authored by five widely used autonomous coding agents on GitHub. The results may not generalize to proprietary development environments, non-GitHub platforms, or organizations with stricter security governance or different review cultures. In addition, both agent capabilities and reviewer practices are evolving rapidly; replication on future datasets will be necessary to assess the stability of the observed patterns over time.

5.6.4. Conclusion Validity

Threats to conclusion validity arise from statistical uncertainty and modeling choices. Large sample sizes may render small effects statistically significant; therefore, we emphasize effect direction, magnitude, and qualitative patterns alongside statistical tests. Our rejection-prediction models are exploratory and rely solely on early observable signals; their moderate performance underscores the complexity and context-dependence of reviewer decision-making in security-related PRs and cautions against overinterpretation.

Claude Code’s small footprint ($n = 67$ confirmed security PRs, statistical

power = 0.64 for the merge-rate comparison) falls below the conventional 0.80 threshold, meaning agent-specific estimates may be imprecise despite large observed effect sizes. We treat Claude Code figures as a preliminary snapshot reflecting its recent public release (2025-Q2-Q3).

6. Conclusion

In this paper, we presented the first large-scale empirical study of security-relevant Agentic pull requests in real-world GitHub repositories. By analyzing security-related PRs authored by five autonomous coding agents, we show that security-related work constitutes a meaningful but minority share of agent activity and is often expressed through supportive security hardening rather than narrowly scoped vulnerability fixes. We find that security-related Agentic-PRs receive heightened human scrutiny, reflected in lower merge rates and substantially longer review latency than non-security PRs, with notable variation across agents, ecosystems, and types of code changes. Our results further indicate that PR rejection is more strongly associated with complexity and verbosity than with explicit security topics. Trained text models also outperform both structured-feature and prompting-based LLM baselines for rejection prediction, suggesting that reviewer judgments are more strongly reflected in the semantic content of PR text than in lightweight metadata or zero-/few-shot prompting alone. Together, these findings highlight that the effectiveness of AI teammates in security-critical workflows depends not only on technical capability but also on alignment with human review practices and expectations in GenAI-enabled software systems. In future work, we will assess the ground-truth security quality of merged agent-authored PRs to determine whether accepted changes actually improve security or introduce new vulnerabilities. Incorporating finer-grained complexity signals, such as lines changed and files affected, would help isolate reviewer scrutiny from agent task self-selection, and replication on future datasets will be needed to assess whether the observed patterns remain stable as agent capabilities evolve. Finally, controlled experiments or reviewer surveys could establish the causal mechanisms underlying the heightened scrutiny observed here and assess whether it is normatively warranted.

7. Declarations

7.1. Funding

Not applicable.

7.2. Ethical Approval

Not applicable.

7.3. Informed Consent

Not applicable.

7.4. Author Contributions

M. L. Siddiq: Conceptualization, methodology design, data collection, data analysis, writing, and editing.

X. Zhao: Data analysis, validation, writing—review and editing.

V. C. Lopes: Data analysis, validation, writing—review and editing.

B. Casey: Data analysis, validation, writing—review and editing.

J. C. S. Santos: Supervision, project administration, data analysis, writing — review, and editing.

7.5. Data Availability

The replication package is available at [61].

7.6. Conflict of Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

7.7. Clinical Trial Number

Not applicable.

References

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, *Advances in Neural Information Processing Systems* 33 (2020) 1877–1901.

- [2] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback, arXiv preprint arXiv:2203.02155 (2022).
- [3] OpenAI, Gpt-4 technical report, arXiv preprint arXiv:2303.08774 (2023).
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 30, Curran Associates, Inc.
- [5] H. Li, H. Zhang, A. E. Hassan, The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering, arXiv preprint arXiv:2507.15003 (2025). doi:10.48550/arXiv.2507.15003.
- [6] A. E. Hassan, H. Li, D. Lin, B. Adams, T.-H. Chen, Y. Kashiwa, D. Qiu, Agentic software engineering: Foundational pillars and a research roadmap, arXiv preprint arXiv:2509.06216 (2025). doi:10.48550/arXiv.2509.06216.
- [7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models (2023). arXiv:2210.03629.
URL <https://arxiv.org/abs/2210.03629>
- [8] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, G. Neubig, Webarena: A realistic web environment for building autonomous agents (2024). arXiv:2307.13854.
URL <https://arxiv.org/abs/2307.13854>
- [9] M. Watanabe, H. Li, Y. Kashiwa, B. Reid, H. Iida, A. E. Hassan, On the use of agentic coding: An empirical study of pull requests on github, arXiv preprint arXiv:2509.14745 (2025). doi:10.48550/arXiv.2509.14745.

- [10] K. Jie, X. Ke, Z. Lyu, A domain-level empirical characterization of ai-authored pull requests in open-source software engineering (2025).
- [11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating large language models trained on code (2021). `arXiv:2107.03374`.
URL <https://arxiv.org/abs/2107.03374>
- [12] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, Y. Yang, W. Sun, S. Yu, Z. Chen, A survey on large language models for software engineering (2024). `arXiv:2312.15223`.
URL <https://arxiv.org/abs/2312.15223>
- [13] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions, in: IEEE Symposium on Security and Privacy, San Francisco, CA, 2022.
URL <https://arxiv.org/abs/2108.09293>
- [14] M. L. Siddiq, S. H. Majumder, M. R. Mim, S. Jajodia, J. C. S. Santos, An empirical study of code smells in transformer-based code generation techniques, in: 2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM), 2022, pp. 71–82. doi: 10.1109/SCAM55253.2022.00014.
- [15] E. Allegrini, A. Shreekumar, Z. B. Celik, Formalizing the safety, security, and functional properties of agentic ai systems, `arXiv preprint arXiv:2510.14133` (2025). doi:10.48550/arXiv.2510.14133.
- [16] D. Moshkovich, S. Zeltyn, Taming uncertainty via automation: Observing, analyzing, and optimizing agentic ai systems, `arXiv preprint arXiv:2507.11277` (2025). doi:10.48550/arXiv.2507.11277.

- [17] M. Li, Y. Zhao, W. Zhang, S. Li, W. Xie, S.-K. Ng, T.-S. Chua, Y. Deng, Knowledge boundary of large language models: A survey (2025) 5131–5157doi:10.18653/v1/2025.acl-long.256.
URL <https://aclanthology.org/2025.acl-long.256/>
- [18] F. Wang, B. Do, J. Jermier, Automated vs. human security patching patterns in pull requests: Evidence from the adev dataset, student project report (University of Waterloo).
URL <https://plg.uwaterloo.ca/~migod/846/current/projects/04-FelixJacieBrian-report.pdf>
- [19] M. L. Siddiq, J. C. S. Santos, S. Devareddy, A. Muller, Sallm: Security assessment of generated code, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW '24), ASEW '24. doi:10.1145/3691621.3694934.
- [20] B. Chatterjee, D. Zagieboylo, S. Damani, S. Hari, C. Kozyrakis, Proofwright: Towards agentic formal verification of cuda, arXiv preprint arXiv:2511.12294 (2025). doi:10.48550/arXiv.2511.12294.
- [21] H. Li, H. Zhang, A. E. Hassan, The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering, arXiv preprint arXiv:2507.15003 (2025).
- [22] R. Hoda, Toward agentic software engineering beyond code: Framing vision, values, and vocabulary, arXiv preprint arXiv:2510.19692 (2025). doi:10.48550/arXiv.2510.19692.
- [23] R. Sapkota, K. I. Roumeliotis, Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai, arXiv preprint arXiv:2505.19443 (2025). doi:10.48550/arXiv.2505.19443.
- [24] F. Zambonelli, A. Omicini, Challenges and research directions in agent-oriented software engineering, *Autonomous Agents and Multi-Agent Systems* 9 (3) (2004) 253–283. doi:10.1023/B:AGNT.0000038028.66672.1e.
- [25] E. Bandara, R. Gore, X. Liang, S. Rajapakse, et al., Agentsway – software development methodology for ai agents-based teams, arXiv preprint arXiv:2510.23664 (2025). doi:10.48550/arXiv.2510.23664.

- [26] Z. Rasheed, M. A. Sami, K.-K. Kemell, et al., Codepori: Large-scale system for autonomous software development using multi-agent technology, arXiv preprint arXiv:2402.01411 (2024). doi:10.48550/arXiv.2402.01411.
- [27] M. A. Sami, M. Waseem, Z. Rasheed, M. Saari, K. Systä, P. Abrahamsson, Experimenting with multi-agent software development: Towards a unified platform, arXiv preprint arXiv:2406.05381 (2024). doi:10.48550/arXiv.2406.05381.
- [28] H. Wang, J. Gong, H. Zhang, J. Xu, Z. Wang, Ai agentic programming: A survey of techniques, challenges, and opportunities, arXiv preprint arXiv:2508.11126 (2025). doi:10.48550/arXiv.2508.11126.
- [29] B. Klieger, C. Charitsis, M. Suzara, S. Wang, N. Haber, J. C. Mitchell, Chatcollab: Exploring collaboration between humans and ai agents in software teams, arXiv preprint arXiv:2412.01992 (2024). doi:10.48550/arXiv.2412.01992.
- [30] K. Ronanki, Facilitating trustworthy human-agent collaboration in llm-based multi-agent system oriented software engineering, in: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, 2025, pp. 1333–1337.
- [31] M. A. Akbar, A. A. Khan, M. Hamza, A. Ghaffar, A. Hajikhani, Agentic ai in software engineering: Practitioner perspectives across the software development life cycle, SSRN (2025). doi:10.2139/ssrn.5520159.
- [32] S. K. Sarkar, Ai agents, productivity, and higher-order thinking: Early evidence from software development, SSRN (2025). doi:10.2139/ssrn.5713646.
- [33] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, A. E. Hassan, An empirical study of testing practices in open source ai agent frameworks and agentic applications, arXiv preprint arXiv:2509.19185 (2025). doi:10.48550/arXiv.2509.19185.
- [34] G. Sandoval, H. Pearce, T. Nys, R. Karri, B. Dolan-Gavitt, S. Garg, Security implications of large language model code assistants: A user study, arXiv preprint arXiv:2208.09727 (2022).

- [35] H. Hajipour, T. Holz, L. Schönherr, M. Fritz, Systematically finding security vulnerabilities in black-box code generation models, arXiv preprint arXiv:2302.04012 (2023).
- [36] M. L. Siddiq, B. Casey, J. C. S. Santos, Franc: A lightweight framework for high-quality code generation, in: 24th IEEE International Conference on Source Code Analysis and Manipulation (SCAM), 2024. doi:10.1109/SCAM63643.2024.00020.
- [37] M. L. Siddiq, J. C. S. Santos, Securityeval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques, in: Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security (MSR4P&S22), 2022. doi:10.1145/3549035.3561184.
- [38] M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana, et al., Purple llama cyberseceval: A secure coding benchmark for language models, arXiv preprint arXiv:2312.04724 (2023).
- [39] S. McIntosh, Y. Kamei, B. Adams, A. E. Hassan, The impact of code review coverage and code review participation on software quality: A case study of the qt, vtk, and itk projects, in: Proceedings of the 11th working conference on mining software repositories, 2014, pp. 192–201.
- [40] P. C. Rigby, D. M. German, L. Cowen, M.-A. Storey, Peer review on open-source software projects: Parameters, statistical models, and theory, ACM Transactions on Software Engineering and Methodology (TOSEM) 23 (4) (2014) 1–33.
- [41] O. Baysal, O. Kononenko, R. Holmes, M. W. Godfrey, Investigating technical and non-technical factors influencing modern code review, Empirical Software Engineering 21 (3) (2016) 932–959.
- [42] A. Ram, A. A. Sawant, M. Castelluccio, A. Bacchelli, What makes a code change easier to review: an empirical investigation on code change reviewability, in: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2018, Association for Computing Machinery, New York, NY, USA, 2018, p. 201–212. doi:

10.1145/3236024.3236080.

URL <https://doi.org/10.1145/3236024.3236080>

- [43] X. Zhang, Y. Yu, T. Wang, A. Rastogi, H. Wang, Pull request latency explained: an empirical overview, *Empirical Softw. Engg.* 27 (6) (Nov. 2022). doi:10.1007/s10664-022-10143-4.
URL <https://doi.org/10.1007/s10664-022-10143-4>
- [44] V. Lenarduzzi, V. Nikkola, N. Saarimäki, D. Taibi, Does code quality affect pull request acceptance? an empirical study, *Journal of Systems and Software* 171 (2021) 110806. doi:<https://doi.org/10.1016/j.jss.2020.110806>.
URL <https://www.sciencedirect.com/science/article/pii/S0164121220302090>
- [45] X. Zhang, Y. Yu, G. Gousios, A. Rastogi, Pull request decision explained: An empirical overview (2021). [arXiv:2105.13970](https://arxiv.org/abs/2105.13970).
URL <https://arxiv.org/abs/2105.13970>
- [46] H. Rebatchi, T. F. Bissyandé, N. Moha, Dependabot and security pull requests: large empirical study, *Empirical Softw. Engg.* 29 (5) (Jul. 2024). doi:10.1007/s10664-024-10523-y.
URL <https://doi.org/10.1007/s10664-024-10523-y>
- [47] H. Schroeder, D. Roy, J. Kabbara, Just put a human in the loop? investigating llm-assisted annotation for subjective tasks, in: *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 25771–25795.
- [48] J. Cohen, A coefficient of agreement for nominal scales, *Educational and Psychological Measurement* 20 (1) (1960) 37–46.
- [49] M. Simion, M. van der Horst, S. Plasmeijer, Y. Zhauniarovich, Beyond CVEs: Untracked vulnerabilities in public issue trackers, in: *Proceedings of the 19th European Workshop on Systems Security (EuroSec '26)*, ACM, 2026, pp. 46–52. doi:10.1145/3803525.3804993.
- [50] J. Ayala, Y.-J. Tung, J. Garcia, A mixed-methods study of open-source software maintainers on vulnerability management and platform security features, in: *Proceedings of the 34th USENIX Security Symposium (USENIX Sec '25)*, USENIX Association, 2025.

- [51] D. Mu, A. Cuevas, L. Yang, H. Hu, X. Xing, B. Mao, G. Wang, Understanding the reproducibility of crowd-reported security vulnerabilities, in: Proceedings of the 27th USENIX Security Symposium (USENIX Sec '18), USENIX Association, 2018.
- [52] A. Strauss, J. Corbin, Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory, 2nd Edition, Sage Publications, Thousand Oaks, CA, 1998.
- [53] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.
- [54] D. W. Hosmer, S. Lemeshow, R. X. Sturdivant, Applied Logistic Regression, 3rd Edition, Wiley, 2013.
- [55] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.
- [56] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, Information Processing & Management 24 (5) (1988) 513–523. doi:10.1016/0306-4573(88)90021-0.
- [57] T. Joachims, Text categorization with support vector machines: Learning with many relevant features, in: Proceedings of ECML, 1998, pp. 137–142. doi:10.1007/BFb0026683.
- [58] A. Joulin, E. Grave, P. Bojanowski, T. Mikolov, Bag of tricks for efficient text classification, arXiv preprint arXiv:1607.01759 (2017).
- [59] V. Sanh, L. Debut, J. Chaumond, T. Wolf, Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, in: NeurIPS Workshop on Energy Efficient Machine Learning and Cognitive Computing, 2019, arXiv:1910.01108.
- [60] OpenAI, GPT-4o mini: Advancing cost-efficient intelligence, <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, accessed: 2026-05-25 (Jul. 2024).

- [61] M. L. Siddiq, X. Zhao, V. C. Lopes, B. Casey, J. C. da Silva Santos, Security in the age of ai teammates: An empirical study of agentic pull requests on github (Dec. 2025). doi:10.5281/zenodo.18103932.
URL <https://doi.org/10.5281/zenodo.18103932>